Cutting the Cord: Trust Without the TA

Security Analysis and Improvements of a TUAV-Assisted Key Management Scheme for Infrastructure-Less IoVs

CS8.501 – Research in Information Security
Project Report

Submitted by:

Swaraj Kumar	Roll Number: 2025202016
Hritik Ranjan	Roll Number: 2025202031
Harsh Raj	Roll Number: 2025202005

Department: Computer Science and Information Security

Date of Submission: November 17, 2026



International Institute of Information Technology, Hyderabad

Abstract

This report presents our course project on the paper "Secure and Efficient Authenticated Key Management Scheme for UAV-Assisted Infrastructure-Less IoVs" by H. Tan, W. Zheng and P. Vijayakumar (IEEE T-ITS, 2023) [1]. The paper replaces broken roadside units in a disaster area with a *tethered* UAV (TUAV) that is powered by a fire truck (ECRV), and uses a trusted authority (TA), reached by satellite, to authenticate a group of UAVs and give them a group key built with the Chinese Remainder Theorem (CRT).

We first study the paper in detail: its motivation, system model, the five protocol phases (IS, US, GA, GD, DU), and its security and performance claims. We then check those claims against the paper's own equations. We prove three weaknesses: the TA can sign as any UAV (F1), the CRT group key leaks every member's key – even to a passive outsider who compares two broadcasts (F2), and one leaked long-term secret opens all past group keys (F3). We then propose four contributions: authentication with the TA offline and a separately sealed key for every member (C1), handover between TUAVs (C2), voting out a compromised TUAV (C3) and splitting the TUAV's key across its tether (C4). Each result is stated as a theorem with a proof and is illustrated with a diagram.

Contents

Abstract **1**

1 Introduction **5**
- 1.1 About this project . . . 5
- 1.2 Problem statement . . . 5
- 1.3 Summary of our work . . . 5
- 1.4 Team and individual contributions . . . 6
- 1.5 Organisation of the report . . . 6

2 Mathematical background **6**
- 2.1 Notation . . . 7
- 2.2 Groups, pairings and hard problems . . . 7
- 2.3 Four named theorems we use . . . 7
- 2.4 Signatures, forking and batch verification . . . 8
- 2.5 Security properties we test . . . 9
- 2.6 Adversary models . . . 9

3 The base paper at a glance **9**
- 3.1 Motivation given by the paper . . . 9
- 3.2 Contributions claimed by the paper . . . 10
- 3.3 Related work discussed in the paper . . . 10
- 3.4 Entities of the system . . . 11
- 3.5 Security objectives stated by the paper . . . 11

4 The base scheme in detail **11**
- 4.1 System model . . . 12

4.2 The protocol at a glance . . . 13
4.3 Initialisation (IS) . . . 14
4.4 UAV signing (US) and group authentication (GA) . . . 14
4.5 Group key distribution (GD) and dynamic update (DU) . . . 15
4.6 What the paper claims . . . 16

5 Security and performance analysis in the paper 16
5.1 Security analysis in the paper . . . 17
5.2 Performance analysis in the paper . . . 17
5.3 Open questions we took forward . . . 18

6 Our analysis and contributions: overview 18
6.1 Our approach . . . 18
6.2 Results at a glance . . . 19

7 F1: the TA can impersonate any UAV 19
7.1 Who knows what . . . 19
7.2 The theorem . . . 20
7.3 Consequences . . . 21

8 F2: the CRT group key leaks – to members and to outsiders 21
8.1 One value is the same for everybody . . . 21
8.2 The insider attack: F2(a) and F2(b) . . . 22
8.3 The passive outsider attack: F2(c) . . . 22
8.4 How this contradicts the paper . . . 24
8.5 Root cause, and why a small patch is not enough . . . 24

9 F3: no forward secrecy 25
9.1 The theorem . . . 25

10 C1: TA-offline authentication with per-member key delivery 26
10.1 Design goals . . . 26
10.2 Keys: certificateless, so the TA cannot sign . . . 27
10.3 Joining a TUAV: UA1–UA5 . . . 27
10.4 Rekeying on every change . . . 29
10.5 Correctness . . . 29
10.6 Security . . . 29
10.7 How C1 answers the three flaws . . . 30
10.8 Cost, computed with the base paper's own constants . . . 30

11 C2: handover between TUAVs 32
11.1 The gap in the base paper . . . 32
11.2 Two ways to move . . . 32
11.3 The protocol HO0–HO4 . . . 32
11.4 Why each part is there . . . 33
11.5 Security . . . 34

11.6 Cost and latency . . . 34

12 C3: evicting a compromised TUAV by quorum **35**
12.1 Why it is needed . . . 35
12.2 Votes and certificates . . . 35
12.3 The protocol . . . 36
12.4 Security . . . 36
12.5 Cost . . . 37

13 C4: the tether-split key **37**
13.1 Why we add it: the TUAV is the new weak spot . . . 38
13.2 The construction . . . 38
13.3 Correctness and security . . . 39
13.4 When the tether is cut . . . 40
13.5 Cost with the base paper's constants . . . 40
13.6 What is known and what is new . . . 40

14 A1–A7: attacking our own design **41**
14.1 Why each fix works . . . 41

15 Results: security comparison and cost **42**
15.1 Security properties . . . 42
15.2 Attacks and their outcome . . . 43
15.3 Cost . . . 43
15.4 What we claim, and what we do not . . . 43

16 Related work **44**

17 Conclusion and future work **45**

References **45**

List of Figures

3.1 Motivation of the base paper. Left: in a normal IoV, vehicles communicate through a roadside unit (RSU). Middle: after a disaster the RSU is down and the vehicle-to-infrastructure link is lost. Right: the paper's idea – a tethered UAV (TUAV), powered through a cable by a ground vehicle (ECRV), acts as the base station, and battery UAVs extend its coverage. . . . 10

4.1 System model of the TUAV-assisted infrastructure-less IoV, reproduced from Fig. 1 of the base paper [1]. The UAV network (top left) talks to the tethered UAV, which is powered through a cable by the ECRV. The ECRV reaches the trusted authority (TA) through the satellite network. Vehicles (bottom left) communicate through the UAV network because the roadside unit (RSU) is out of service. . . . 12

4.2 The five phases of the base scheme, in order. The step labels on the left (IS, US, GA, GD, DU) are the ones used in Section IV of the paper. Solid arrows show the normal order; the dashed arrow shows that a key update repeats the key-distribution computation. 13

4.3 Message flow of the base scheme between a UAV $\mathcal{U}_i$, the TUAV and the TA. Time runs from top to bottom. Solid arrows are local radio messages; dashed arrows cross the satellite link. Boxes show the local computation of each entity. 14

5.1 Group-authentication (GA) time against the number of UAVs, plotted from the cost formulas in Section VI-A of the base paper for the four schemes it compares with (Kumar et al., Xu et al., Mei et al., Ali et al.) and for itself. The base paper's line counts only the TUAV side; the TA's pairings are not included. 18

6.1 How our results fit together. Left: the three weaknesses we prove in the base scheme. They are all removed by contribution C1. After C1 the TA is offline, so the TUAV becomes the trusted party; this leaves three gaps (third column), which are closed by C2, C3 and C4. A1–A7 are improvements to our own design. 19

7.1 Flowchart of the TA-forgery attack (Algorithm 7.1). Starting from the stored pair $\langle id_i, s_i \rangle$, the forger builds a request with a public ξ'; the honest TUAV and TA then accept it. No step uses a value chosen by the real UAV. 20

8.1 Flowchart of the insider attack (Theorem 8.2). A member evaluates the broadcast polynomial at its own key, subtracts the result and finds the roots; this gives every member's key. Depending on what happens later, the same member can break revocation, F2(a), or backward secrecy, F2(b). 22

8.2 Flowchart of the passive outsider attack (Algorithm 8.1). The attacker records the coefficient sets before and after one membership change, obtains the changed member's key from the two coefficient sums, removes the mask, and finds all keys and both group keys. It needs no registration and no secret. 23

9.1 Timeline of the forward-secrecy failure. In each past session the attacker records the public values $\xi_i^{(t)}$ and $\{a_k^{(t)}\}$. When the long-term secret s_i leaks later, the attacker recomputes the group key of every recorded session (dashed arrows). 26

10.1 Overview of contribution C1. Left: before the mission, the TA sets up the system and issues one certificateless credential to every UAV and TUAV. Right: during the mission the TA is offline; a UAV joins its TUAV in one local exchange (steps UA1–UA5), the TUAV verifies requests using only the TA's public key P_{pub}, and each member receives the group key sealed under its own key. 27

10.2 Message flow of C1 between the TUAV and n UAVs. Time runs from top to bottom; boxes show local computation. The TA (centre) is never contacted. Compared with Figure 4.3, there is no satellite link, no pairing and no CRT polynomial. 28

10.3 Traffic needed to distribute one group key against the number of UAVs n (logarithmic vertical axis), computed from the message formats. The base scheme sends all polynomial coefficients to each UAV, about $10n^3$ bytes in total; C1 sends one broadcast with about 80 bytes per member. The marked points are the values for $n = 120$. . . 31

11.1 Message flow of the C2 handover between the old TUAV $\mathcal{T}_a$, the moving member $\mathcal{M}$ and the new TUAV $\mathcal{T}_b$. Time runs from top to bottom. The two signatures written in bold (by the member in HO2 and by the new TUAV in HO3) are the fixes A2 and A7 of Chapter 14. 33

12.1 Quorum eviction (C3). The neighbours T_1, T_2, T_3 obtain evidence against the compromised TUAV T_7 (dashed arrows) and each signs a vote. At least t distinct votes form an eviction certificate, which is delivered to the members of T_7; they verify it and re-join a neighbour. 36

12.2 Flowchart of how a member checks an eviction certificate against the four conditions V1–V4 of Definition 12.1. Only if all conditions hold does the member leave its TUAV. 36
13.1 The tether-split key (C4). The drone holds the share a and a secure chip in the ECRV holds the share g; the two are connected by the tether. The public key $pk = aP + gP$ is unchanged, and no single device holds the full key $a + g$. 38
13.2 Joint signing over the tether (steps J1–J5). Messages ①–④ travel on the tether between the drone and the chip; the final signature (R, v) is sent by radio and is verified with the usual check. The commitment in ① forces the drone to fix its nonce before it sees the chip's. 39
13.3 Timeline after the tether is cut. The chip stops signing at once, the ECRV broadcasts the stored kill notice, old beacons expire after the period δ, and members re-join a neighbouring TUAV with fresh keys. 40

CHAPTER 1

Introduction

1.1 About this project

This is the project report for the course **CS8.501 – Research in Information Security**. The project is based on one research paper:

> H. Tan, W. Zheng and P. Vijayakumar, "Secure and Efficient Authenticated Key Management Scheme for UAV-Assisted Infrastructure-Less IoVs," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 6, pp. 6389–6400, June 2023 [1].

1.2 Problem statement

After an earthquake or a flood, roadside units (RSUs) and mobile towers often stop working. Rescue vehicles still need to communicate, and the communication must be authenticated and confidential. The base paper uses a **tethered UAV (TUAV)**, a drone powered through a cable by an emergency vehicle (ECRV), as a flying base station. Battery UAVs extend its coverage, and a trusted authority (TA) is reached over a satellite link. The TA authenticates a group of UAVs, and the TUAV gives them a group key.

Our problem has two parts: (i) check whether the scheme really gives the security properties it claims, and (ii) where it does not, design improvements that keep the system working securely in a disaster area.

1.3 Summary of our work

Part	What we did
Study of the paper	We studied the base paper in detail: the problem it solves, its system model, the protocol step by step, and its security and performance claims (Chapters 2–5).
Analysis and design	We checked every security claim against the paper's own equations, proved three weaknesses (F1–F3), and designed four contributions (C1–C4) that fix them and cover situations the paper does not handle (Chapters 6–17).

1.4 Team and individual contributions

All three members studied the base paper, took part in the discussions and reviewed the final report. The main responsibilities were:

Member	Roll number	Main responsibilities
Swaraj Kumar	2025202016	study of the paper's motivation, system model and protocol; overall framework of our design; compiling the report
Hritik Ranjan	2025202031	mathematical analysis of the three weaknesses (F1, F2, F3); construction of C1 and its security theorems
Harsh Raj	2025202005	design and security analysis of C2, C3 and C4; the hardening study A1–A7; security comparison and results

1.5 Organisation of the report

Chapter 2 gives the mathematical background. Chapter 3 summarises the paper's motivation, contributions and related work. Chapter 4 describes the protocol, and Chapter 5 summarises the paper's own security and performance analysis. Chapter 6 gives an overview of our results. Chapters 7–9 prove the three weaknesses, Chapters 10–14 present our contributions, and Chapters 15–17 compare, discuss and conclude.

CHAPTER 2

Mathematical background

This chapter collects every tool used later. Where a result has a standard name we use it, so a reader can look it up.

2.1 Notation

Symbol	Meaning
$\mathbb{G}_1, \mathbb{G}_2,\ \hat{e},\ P,\ q$	base paper: additive group $\mathbb{G}_1$ and multiplicative group $\mathbb{G}_2$ of prime order q, pairing $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_1 \to \mathbb{G}_2$, generator P
$\langle id_i, s_i \rangle$	base paper: true identity and long-term secret of UAV i, issued and *stored* by the TA (IS2)
$r_i,\ \xi_i = h_1(r_i)$	base paper: UAV's random value and its hash; ξ_i is sent in the clear (US3)
$ID_i,\ \upsilon_i,\ \Gamma_i$	base paper: session identity, check value and credential of UAV i (US1–US2)
$\sigma_i = h_1(s_i \xi_i) P$	base paper: "decrypting key" of UAV i, used as a CRT modulus (GD1)
$\Lambda(x) = \sum_{k=0}^{n} a_k x^k$	base paper: broadcast polynomial; $\{a_k\}$ is the public coefficient set (GD2, GD4)
κ_{tu}	base paper: group key
$\mathbb{G},\ P,\ q$	our design: a prime-order elliptic-curve group (no pairing needed)
$s,\ P_{\text{pub}} = sP$	our design: TA master key and public key (used only before the mission)
$x,\ d,\ sk = x + d,\ pk$	our design: user secret, partial key from the TA, full signing key, public key
$e,\ E = eP$	ephemeral Diffie–Hellman values, erased after use
$GK,\ \nu$	our group key and its version number
KDF, Enc	key-derivation function (e.g. HKDF [16]) and authenticated encryption with associated data (AEAD [17])
$a,\ g$	C4: the drone's share and the ECRV chip's share of the TUAV key, $sk = a + g$

2.2 Groups, pairings and hard problems

The base paper uses a symmetric pairing $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_1 \to \mathbb{G}_2$ with **bilinearity**

$$\hat{e}(aP, bQ) = \hat{e}(P, Q)^{ab}, \qquad \hat{e}(Q + R, P) = \hat{e}(Q, P)\, \hat{e}(R, P),$$

non-degeneracy and efficient computability (paper, Section III-A). Security rests on two problems, defined in the paper as Definitions 1 and 2.

Definition 2.1 (ECDLP)**.** Given P and $Q = aP$, find a. The problem is hard if every probabilistic polynomial-time (PPT) algorithm succeeds with negligible probability.

Definition 2.2 (CDH)**.** Given P, aP and bP for unknown a, b, compute abP. Hard if every PPT algorithm succeeds with negligible probability.

A CDH *instance* is only hard if a and b are unknown; if b is public, anyone computes $abP = b \cdot (aP)$.

2.3 Four named theorems we use

Theorem 2.3 (Chinese Remainder Theorem)**.** *Let $m_1, \ldots, m_n$ be pairwise coprime positive integers and $M = \prod_i m_i$. For any $c_1, \ldots, c_n$ there is exactly one $X \in [0, M)$ with $X \equiv c_i \pmod{m_i}$ for all i, namely $X = \sum_i c_i\, \omega_i \varphi_i \bmod M$, where $\omega_i = M/m_i$ and $\varphi_i \equiv \omega_i^{-1} \pmod{m_i}$.*

The base paper uses the special case $c_1 = \cdots = c_n = \kappa_{tu}$: the number $\kappa_{tu} \sum_i \omega_i \varphi_i$ leaves remainder κ_{tu} modulo every σ_i (Lemma 4.2).

Theorem 2.4 (Vieta's formulas)**.** *If $p(x) = \prod_{i=1}^{n}(x - \sigma_i) = x^n + c_{n-1}x^{n-1} + \cdots + c_1 x + c_0$, then $c_{n-k} = (-1)^k e_k(\sigma_1, \ldots, \sigma_n)$, where e_k is the k-th elementary symmetric polynomial. In particular*

$$c_{n-1} = -(\sigma_1 + \sigma_2 + \cdots + \sigma_n), \qquad c_0 = (-1)^n \sigma_1 \sigma_2 \cdots \sigma_n.$$

Theorem 2.5 (Rational Root Theorem)**.** *If a polynomial with integer coefficients has a rational root u/v in lowest terms, then u divides the constant coefficient and v divides the leading coefficient. For a monic polynomial every rational root is an integer.*

Proposition 2.6 (Integer roots are easy to find)**.** *All integer roots of an integer polynomial of degree n with B-bit coefficients can be found in time polynomial in n and B – for example by factoring with the LLL algorithm [14], or by Cantor–Zassenhaus factoring modulo a prime followed by Hensel lifting [15]. When all roots are real and distinct (our case), real-root isolation followed by bisection finds each root of size at most 2^{160} in about 160 halving steps.*

So recovering the σ_i from a known product polynomial is **not** a brute-force search; it is a routine computation.

2.4 Signatures, forking and batch verification

Definition 2.7 (Schnorr-type signature [5])**.** Key pair (sk, $pk = sk \cdot P$). To sign m: pick random r, set $R = rP$, $c = H(m \,\|\, R \,\|\, \ldots)$, $v = r + c \cdot sk \bmod q$; output (R, v). To verify: check $vP \stackrel{?}{=} R + c \cdot pk$.

Lemma 2.8 (General Forking Lemma [6, 7], informal)**.** *If an adversary making at most q_H hash queries outputs a valid signature with probability ε, then re-running it with the same random tape but fresh hash answers from the critical query yields two valid signatures (R, v), (R, v') with the same R and different challenges $c \neq c'$ with probability at least about ε^2/q_H. From $vP = R + c\,pk$ and $v'P = R + c'\,pk$ one extracts $sk = (v - v')(c - c')^{-1}$, solving the ECDLP for pk.*

Lemma 2.9 (Small-Exponent Test, Bellare–Garay–Rabin [8])**.** *Let $\Delta_1, \ldots, \Delta_n \in \mathbb{G}$ (prime order q) and let $\lambda_1, \ldots, \lambda_n$ be chosen uniformly from $\{1, \ldots, 2^\ell\}$ with $2^\ell < q$. If some $\Delta_j \neq 0$, then $\Pr\left[\sum_i \lambda_i \Delta_i = 0\right] \leq 2^{-\ell}$.*

Proof. Write $\Delta_i = \delta_i P$ with $\delta_j \neq 0$. The sum is zero iff $\sum_i \lambda_i \delta_i \equiv 0 \pmod q$. Fix every λ_i with $i \neq j$; then the equation holds for exactly one value of λ_j modulo q, namely $-\delta_j^{-1} \sum_{i \neq j} \lambda_i \delta_i$. Since λ_j is uniform over $2^\ell < q$ values, it hits that value with probability at most $2^{-\ell}$. □

Applied with $\Delta_i = v_i P - R_i - c_i pk_i$ (the "error" of signature i), the test accepts a batch containing an invalid signature with probability at most $2^{-\ell}$; we use $\ell = 64$. Without weights ($\lambda_i = 1$), errors $+\Delta$ and $-\Delta$ cancel.

2.5 Security properties we test

Definition 2.10 (Key-escrow resilience and Girault's trust levels [9])**.** A scheme is *key-escrow resilient* if the authority that issues credentials cannot produce valid signatures in a user's name. Girault grades authorities as: **level 1** – it knows users' secret keys and can impersonate without trace; **level 2** – it does not know them, but can impersonate by issuing a false credential (a second public key); **level 3** – any such false credential can be proven.

Definition 2.11 (Non-repudiation)**.** A sender cannot later deny a message it signed; equivalently, nobody else (including the authority) could have produced that signature.

Definition 2.12 (Group-key secrecy, revocation and backward secrecy)**.** For a sequence of group keys $\kappa^{(1)}, \kappa^{(2)}, \ldots$: *secrecy* – no non-member learns a current key; *revocation* (forward group secrecy) – a member removed before epoch t learns nothing about $\kappa^{(t)}, \kappa^{(t+1)}, \ldots$; *backward secrecy* – a member that joins in epoch t learns nothing about $\kappa^{(1)}, \ldots, \kappa^{(t-1)}$.

Definition 2.13 (Forward secrecy)**.** Session (or group) keys established before a long-term secret is compromised remain secret after the compromise.

2.6 Adversary models

Adversary	Capabilities	Used in
Passive outsider	records every radio message (Dolev–Yao eavesdropper [26]); never registered	F2(c)
Active outsider	also injects, replays, drops and modifies messages	C1, C2
Insider (current or former member)	knows its own secrets and everything it ever received	F2(a), F2(b)
Malicious or breached TA	knows everything the TA stores ($\langle id_i, s_i \rangle$ in the base paper; s, d_i in ours)	F1, C1
Long-term-key thief	obtains s_i (e.g. from a captured UAV) later in time	F3, C1
Captured TUAV	has the drone's full memory but no longer the tether	C4
Compromised TUAV	software takeover while still tethered	C3, A1–A7

CHAPTER 3

The base paper at a glance

3.1 Motivation given by the paper

The paper starts from three observations (Section I of [1]):

1. Vehicle networks (IoVs) normally rely on roadside units (RSUs) at the edge. Existing authentication schemes assume these RSUs work. In a disaster they may not, and then vehicle-to-infrastructure (V2I) communication fails.
2. Modern roads are three-dimensional (bridges, flyovers, tunnels), so flat ground coverage is not enough. Satellites have high latency and cost, so the paper prefers UAVs as an aerial extension of

the network.

3. Ordinary UAVs fly for less than an hour on battery. A **tethered UAV** gets power through a cable from a ground vehicle, so it can hover for a long time and act as a mobile base station and edge server.

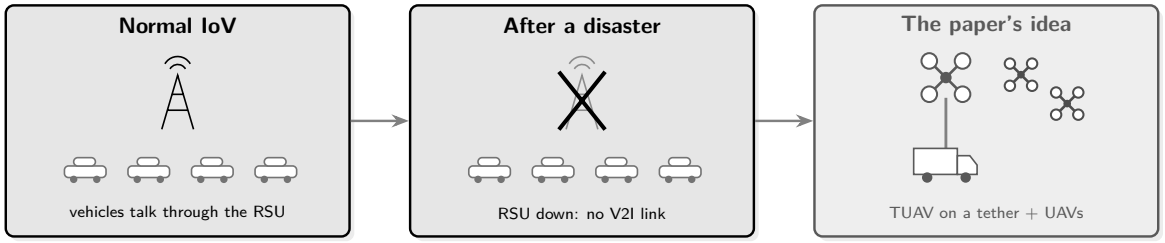


Figure 3.1: Motion of the base paper. Left: in a normal IoV, vehicles communicate through a roadside unit (RSU). Middle: after a disaster the RSU is down and the vehicle-to-infrastructure link is lost. Right: the paper's idea – a tethered UAV (TUAV), powered through a cable by a ground vehicle (ECRV), acts as the base station, and battery UAVs extend its coverage.

3.2 Contributions claimed by the paper

Quoted from the base paper

- "Flexible and dynamic TUAV-enabled edge architecture for infrastructure-less IoV" – the TUAV replaces the failed edge server; the authors state that this is the first use of a TUAV-enabled edge network for infrastructure-less IoV.
- "Secure and efficient TUAV-assisted group authentication mechanism with anonymity" – a certificateless group authentication with batch validation, where "the complex pairing computation tasks are allocated to the cloud server".
- "Dynamic UAV management with updating strategy" – a group key built with the Chinese Remainder Theorem, with low cost for updating.

(Section I-A)

3.3 Related work discussed in the paper

Section II of the paper groups earlier work into a few families:

Family	Examples cited by the paper
Authenticated key management for IoV	dual authentication and key agreement for V2V; certificateless conditional privacy-preserving authentication (CL-CPPA); fog-computing based AKM
Identity- and attribute-based schemes	pairing-free identity-based anonymous authentication with vehicle revocation; attribute-based encryption for vehicular social networks
Lattice-based schemes	conditional privacy-preserving authentication based on the short integer solution problem
Group / batch authentication	batch authentication protocols that verify many vehicles at once, and certificateless aggregate signatures
UAV-assisted IoV	schemes that still need a working local edge infrastructure

The paper's main argument is that none of these works independently of the local edge infrastructure, which is the gap it addresses.

3.4 Entities of the system

Entity	Role in the paper (Section III-D)
TA (trusted authority)	trusted server with large storage; does the setup and the user verification; reached through satellite ("space") networks
TUAV (tethered UAV)	multirotor drone powered through a cable; acts as a substitute edge base station
ECRV	emergency communication response vehicle (a fire truck); powers the TUAV and acts as a control centre
UAV network	battery drones (under one hour of flight) that give 3D coverage
Vehicular network	vehicles that exchange driving data through the UAV network

3.5 Security objectives stated by the paper

The paper lists six goals (Section III-E): **mutual authentication**, **user anonymity**, **unforgeability**, **non-repudiation**, **conditional privacy preservation** (only a law-enforcement agency can reveal a malicious user) and **session key establishment**. Chapter 4 shows how the protocol tries to reach them, and Chapter 5 summarises how the paper argues it does.

CHAPTER 4

The base scheme in detail

Everything in this chapter is taken from Sections III–V of [1]. Equations are copied as printed; where the paper's wording matters we quote it.

4.1 System model

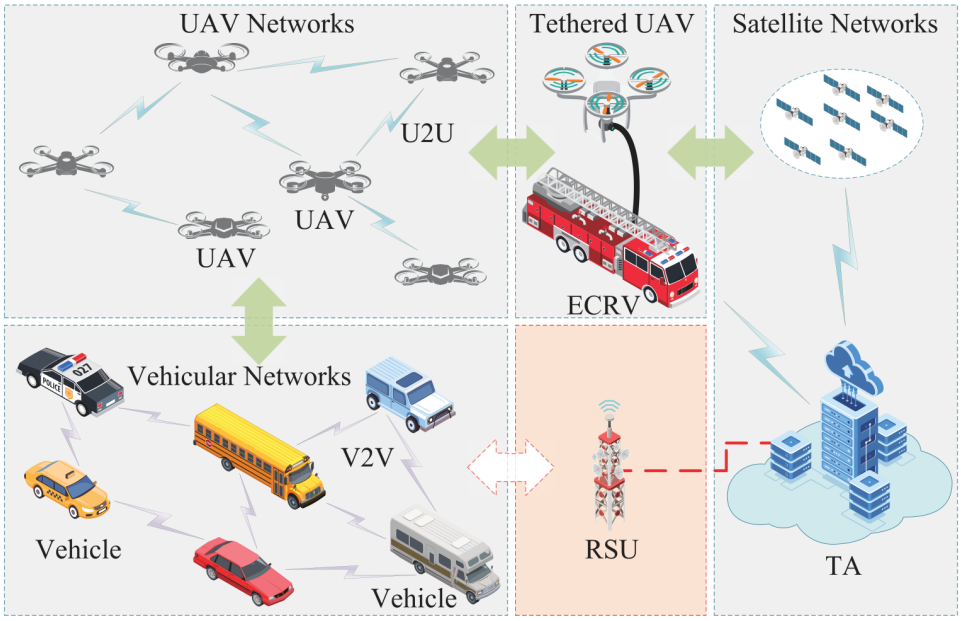


Figure 4.1: System model of the TUAV-assisted infrastructure-less IoV, reproduced from Fig. 1 of the base paper [1]. The UAV network (top left) talks to the tethered UAV, which is powered through a cable by the ECRV. The ECRV reaches the trusted authority (TA) through the satellite network. Vehicles (bottom left) communicate through the UAV network because the roadside unit (RSU) is out of service.

Quoted from the base paper

"the data links between TA and TUAV are conducted with the secure communication channel, which takes the place of the dysfunctional edge RSUs." "the TUAVs utilize flexible tethered wires or cables as the power source instead of conventional batteries. The ECRV, as the auxiliary unit and control center, maintains the wired power supply to the connected TUAV." (Section III-D)

4.2 The protocol at a glance

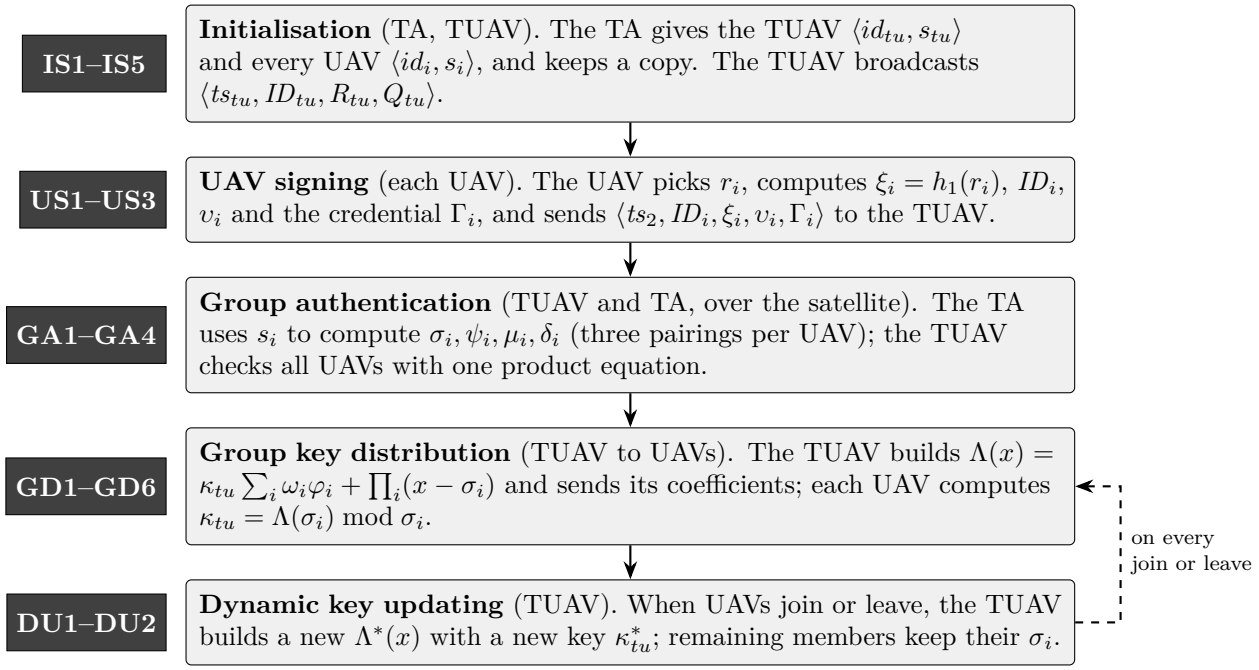


Figure 4.2: The five phases of the base scheme, in order. The step labels on the left (IS, US, GA, GD, DU) are the ones used in Section IV of the paper. Solid arrows show the normal order; the dashed arrow shows that a key update repeats the key-distribution computation.

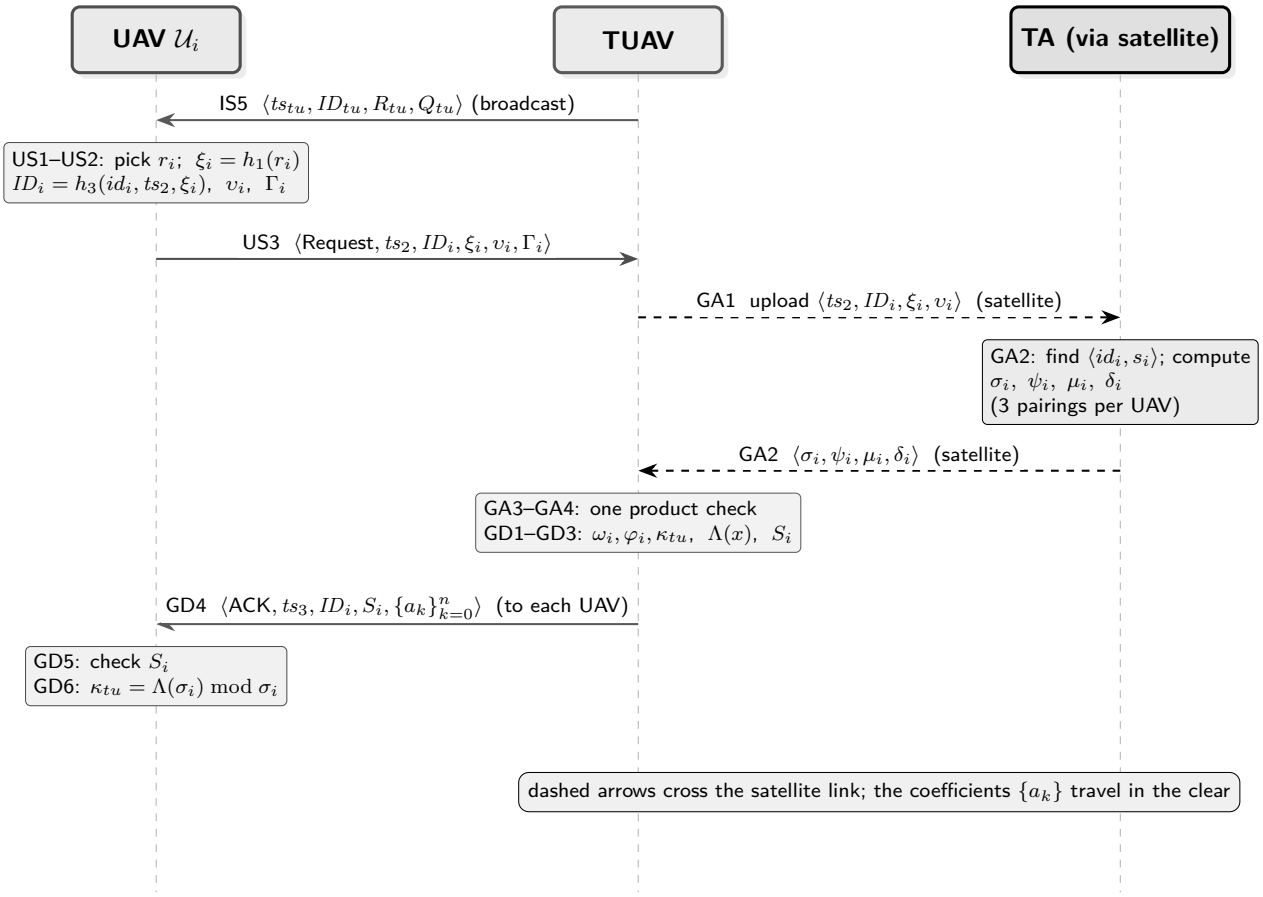


Figure 4.3: Message flow of the base scheme between a UAV $\mathcal{U}_i$, the TUAV and the TA. Time runs from top to bottom. Solid arrows are local radio messages; dashed arrows cross the satellite link. Boxes show the local computation of each entity.

4.3 Initialisation (IS)

IS1–IS5 (Section IV-A)

IS1. TA $\to$ TUAV: $\langle id_{tu}, s_{tu}\rangle$, "safely shared only between TA and TUAV itself during the whole time".
IS2. TA $\to \mathcal{U}_i$: $\langle id_i, s_i\rangle$; "both the TUAV and UAV identity sets are safely stored by TA".
IS3. Groups $\mathbb{G}_1, \mathbb{G}_2$ of prime order q, generator P, pairing $\hat{e}$, hashes $\mathsf{H}_1, h_1, h_2, h_3, h_4$.
IS4. The TUAV, "with the effective range covering the entire region", picks r_{tu} and computes

$$ID_{tu} = h_2(id_{tu}, r_{tu}), \qquad R_{tu} = r_{tu}\, h_2(ts_{tu}, s_{tu}), \qquad Q_{tu} = s_{tu}\, h_2(ID_{tu}, r_{tu})\, P.$$

IS5. The TUAV broadcasts $\langle ts_{tu}, ID_{tu}, R_{tu}, Q_{tu}\rangle$ periodically.

4.4 UAV signing (US) and group authentication (GA)

US1–US3 (Section IV-B1, each UAV)

US1. Pick random r_i; $\xi_i = h_1(r_i)$; $ID_i = h_3(id_i, ts_2, \xi_i)$.
US2. $\upsilon_i = h_4(ID_i, ID_{tu}, ts_2, \xi_i)$ and

$$\Gamma_i = \big[s_i\, h_3(ID_i, ts_2, s_i)\, R_{tu} - h_1(s_i\xi_i)\big]P - \upsilon_i\, h_1(r_i)\, Q_{tu}. \tag{4.1}$$

US3. Send $\langle \text{Request}, ts_2, ID_i, \xi_i, \upsilon_i, \Gamma_i \rangle$ to the TUAV.

GA1–GA4 (Section IV-B2, TUAV and TA)

GA1. TUAV checks ts_2 and υ_i and "uploads" $\langle ts_2, ID_i, \xi_i, \upsilon_i \rangle$ "to the remote TA".
GA2. TA recovers $\langle id_i, s_i \rangle$ for ID_i and computes, for every i,

$$\sigma_i = h_1(s_i\xi_i)P, \qquad \psi_i = \hat{e}(\Gamma_i + h_1(s_i\xi_i)P,\, P),$$
$$\mu_i = \hat{e}(h_3(ID_i, ts_2, s_i)P,\, s_iP), \qquad \delta_i = \hat{e}(s_{tu}P,\, \upsilon_i\xi_iP),$$

and forwards $\langle \sigma_i, \psi_i, \mu_i, \delta_i \rangle_{i\in[1,n]}$ to the TUAV.
GA3. TUAV checks

$$\prod_{i=1}^{n} \psi_i \Big(\prod_{i=1}^{n} \delta_i \Big)^{h_2(ID_{tu}, r_{tu})} \stackrel{?}{=} \prod_{i=1}^{n} \mu_i^{R_{tu}}. \tag{4.2}$$

GA4. "If matches, the validity of the vehicles can be proved."

Lemma 4.1 (Correctness of GA3, the paper's Eq. (1))**.** *For honestly generated requests, Eq.* (4.2) *holds.*

Proof. Write $h_2 = h_2(ID_{tu}, r_{tu})$ and $h_3 = h_3(ID_i, ts_2, s_i)$. By (4.1) and $h_1(r_i) = \xi_i$, $\Gamma_i + h_1(s_i\xi_i)P = s_ih_3R_{tu}P - \upsilon_i\xi_iQ_{tu}$, so by bilinearity $\psi_i = \hat{e}(s_ih_3R_{tu}P, P) \cdot \hat{e}(\upsilon_i\xi_iQ_{tu}, P)^{-1}$. Since $Q_{tu} = s_{tu}h_2P$, $\delta_i^{h_2} = \hat{e}(s_{tu}h_2P, \upsilon_i\xi_iP) = \hat{e}(\upsilon_i\xi_iQ_{tu}, P)$. The two factors cancel and $\psi_i\delta_i^{h_2} = \hat{e}(h_3P, s_iP)^{R_{tu}} = \mu_i^{R_{tu}}$. Multiply over i. □

4.5 Group key distribution (GD) and dynamic update (DU)

GD1–GD6 (Section IV-C)

GD1. TUAV takes the decrypting keys $\sigma_i = h_1(s_i\xi_i)P$, builds $\omega_i = \prod_{j\neq i}\sigma_j$ and $\varphi_i \equiv \omega_i^{-1} \bmod \sigma_i$, and selects the group key $\kappa_{tu} \in \mathbb{Z}_q^*$.
GD2. TUAV constructs

$$\Lambda(x) = \kappa_{tu} \sum_{i=1}^{n} \omega_i\varphi_i + \prod_{i=1}^{n}(x - \sigma_i) = \sum_{k=0}^{n} a_k x^k. \tag{4.3}$$

GD3. $S_i = \mathsf{H}_1(ts_3, ID_i,\, s_{tu}h_2(ID_{tu}, r_{tu})\,\sigma_i)$; the acknowledgement "is delivered one by one to each vehicle".
GD4. Packet $\langle \text{ACK}, ts_3, ID_i, S_i, \{a_k\}_{k\in[0,n]} \rangle$.
GD5. UAV checks $S_i \stackrel{?}{=} \mathsf{H}_1(ts_3, ID_i, h_1(s_i\xi_i)Q_{tu})$.
GD6. UAV rebuilds $\Lambda(x)$ from $\{a_k\}$ and computes $\kappa_{tu} = \Lambda(\sigma_i) \bmod \sigma_i$.

DU1–DU2 (Section IV-D)

DU1. For $\ominus$ removed and $\oplus$ joining UAVs, the TUAV picks κ_{tu}^* and computes $\Lambda^*(x) = \prod_{i=1}^{n-\ominus+\oplus}(x - \sigma_i) + \kappa_{tu}^*\big(\sum_{i=1}^{n-\ominus+\oplus}\omega_i\varphi_i\big)$.
DU2. "all the legitimate UAVs could easily extract κ_{tu}^* with $\kappa_{tu} = \Lambda^*(\sigma_i) \bmod \sigma_i$" – using the *same* σ_i as before.

Lemma 4.2 (Correctness of GD6 by the CRT)**.** *If the σ_i are pairwise coprime and $\kappa_{tu} < \sigma_i$ for all i, then $\Lambda(\sigma_i) \bmod \sigma_i = \kappa_{tu}$ for every member i.*

Proof. At $x = \sigma_i$ the product in (4.3) contains the factor $(\sigma_i - \sigma_i) = 0$, so $\Lambda(\sigma_i) = \kappa_{tu} \sum_j \omega_j \varphi_j$. For $j \neq i$, ω_j contains the factor σ_i, so $\omega_j \varphi_j \equiv 0 \pmod{\sigma_i}$; and $\omega_i \varphi_i \equiv 1 \pmod{\sigma_i}$ by the choice of φ_i (Theorem 2.3). Hence $\Lambda(\sigma_i) \equiv \kappa_{tu} \pmod{\sigma_i}$, and as $\kappa_{tu} < \sigma_i$ the remainder is κ_{tu} itself. □

Remark 4.3 (Point or integer?)*.* The paper writes $\sigma_i = h_1(s_i \xi_i)P$, a curve point, yet uses it as a CRT modulus and as a root of a polynomial over the integers. Any implementation must therefore map σ_i to an integer (for instance a fixed integer encoding of the point, or a prime derived from the same hash), and the coefficients a_k must be kept as integers – otherwise "mod σ_i" in GD6 is undefined. All our results hold for *every* such integer reading; nothing depends on which encoding is chosen.

4.6 What the paper claims

Quoted from the base paper

Section IV-D: "the group key updating mechanism supports multi-device revocation and new device participation ... the dynamic key batch updating process can be securely executed in the open wireless environment since the malicious entities that successfully collect the coefficient set $\{a_i\}_{i \in [0,n]}$ can not acquire the κ_{tu} without the correct decrypting key σ_i."

Proof of Theorem 4: "The certificateless authentication is adopted, where the partial secret set $\langle s_i, s_{tu} \rangle$ of TUAV and UAVs is generated and determined by TA. On the other hand, the UAV itself also randomly generates r_i as the partial key ... "

Proof of Theorem 5: "The remaining legitimate UAVs could always use the previously issued key to decrypt the new κ_{tu} with the updated $\Lambda(x)$, if the σ_i has not been removed by TUAV."

Table II: the proposed scheme is ticked (✓) for all of F1–F11: unforgeability, conditional privacy, session key establishment, **key escrow resilience**, scalability, dynamic identity updating, **forward secrecy**, batch validation, unlinkability, **non-repudiation** and replay attack resilience.

Claim in [1]	Paper	This report
Key escrow resilience (Table II, F4)	✓	✗ fails: the TA can sign for any UAV (F1, Theorem 7.1)
Non-repudiation (Table II, F10)	✓	✗ fails: the TA could have produced any request (F1)
"supports multi-device revocation" (IV-D, Thm. 5)	✓	✗ fails: a removed UAV computes the new key (F2a)
New-device participation is secure (IV-D)	✓	✗ fails: a newcomer reads old keys (F2b)
Coefficients reveal nothing without a σ_i (IV-D)	✓	✗ fails: a passive outsider gets all keys (F2c)
Forward secrecy (Table II, F7)	✓	✗ fails: a leaked s_i opens all past keys (F3)

CHAPTER 5

Security and performance analysis in the paper

5.1 Security analysis in the paper

Section V of the paper gives five theorems. We summarise each one and how the paper argues it.

Theorem (paper)	What it claims and how it is argued
1 – Message unforgeability	secure against chosen-message attacks in the random-oracle model if CDH is hard; the proof embeds the CDH instance $(s_{tu}P,\ (\sum_i \upsilon_i \xi_i)P)$ and uses two forgeries with different h_2 values
2 – User anonymity and unlinkability	temporary identities $ID_i = h_3(id_i, ts_2, \xi_i)$ change every session, so eavesdroppers cannot link messages
3 – Replay attack resistance	every credential includes a fresh timestamp, so old messages fail the current check
4 – Conditional privacy	real identities stay hidden, but the law-enforcement agency can reveal a malicious UAV when the TA reports it
5 – Session key establishment	every verified UAV gets the same group key $\kappa_{tu} = \Lambda(\sigma_i) \bmod \sigma_i$; remaining UAVs can decrypt updated keys "if the σ_i has not been removed by TUAV"

In its Table II the paper compares itself with four earlier schemes on eleven properties (F1–F11) and ticks all eleven for itself, including *key escrow resilience*, *forward secrecy* and *non-repudiation*.

5.2 Performance analysis in the paper

The paper measures basic operations with the MIRACL library at an 80-bit security level: a supersingular curve with a 512-bit field for the pairing group $\mathbb{G}_1$, and an ordinary 160-bit curve for the group $\mathbb{G}$.

Symbol	Operation	Time (ms)
T_P	bilinear pairing	7.2232
$T_{SM \cdot P}$	scalar multiplication in $\mathbb{G}_1$	2.6784
$T_{H \cdot MP \cdot P}$	MapToPoint hash	8.1940
$T_{SM \cdot E}$	scalar multiplication on the ordinary curve	0.9018
$T_{PA \cdot E}$	point addition on the ordinary curve	0.0107
T_{ME}	modular exponentiation	0.6256
T_{SH}	hash	0.0012

With these constants the paper reports: UAV signing (US) $2T_{SM \cdot E} + T_{PA \cdot E} + 5T_{SH} \approx 1.82$ ms, single verification (SA) ≈ 2.16 ms, and group authentication (GA) $1.2524n + 0.9054$ ms, i.e. about 151 ms for $n = 120$ UAVs. For GA the paper notes that the pairing work is done by the TA "prior to the verification of TUAV", so it counts the TUAV side.

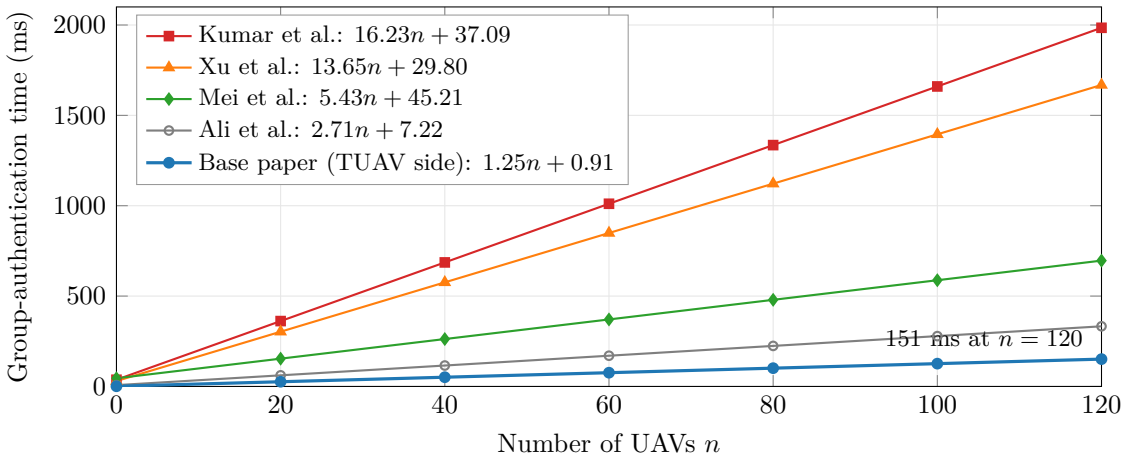


Figure 5.1: Group-authentication (GA) time against the number of UAVs, plotted from the cost formulas in Section VI-A of the base paper for the four schemes it compares with (Kumar et al., Xu et al., Mei et al., Ali et al.) and for itself. The base paper's line counts only the TUAV side; the TA's pairings are not included.

For communication, the paper uses 128-byte elements in $\mathbb{G}_1$, 40-byte elements in $\mathbb{G}$, 20-byte hashes and 4-byte timestamps, and reports lower overhead than the four compared schemes (its Table V).

5.3 Open questions we took forward

While studying the paper we noted several points that we wanted to check carefully:

1. The TA stores every UAV's secret s_i (IS2), yet the paper claims key-escrow resilience and non-repudiation. → F1
2. The coefficients of $\Lambda(x)$ are sent in the clear and only the constant term is masked. → F2
3. The decrypting key $\sigma_i = h_1(s_i\xi_i)P$ uses no fresh secret, yet forward secrecy is claimed. → F3
4. Every login goes through the TA over the satellite, and only one TUAV is assumed for the whole region (IS4). → C1, C2
5. Nothing is said about a TUAV that is hacked or physically captured. → C3, C4

CHAPTER 6

Our analysis and contributions: overview

6.1 Our approach

We treated the paper's printed equations as the source of truth. For every security claim we asked: *what exactly does an attacker know, and what can it compute from that?* When a claim failed, we wrote the failure as a theorem with a proof and a small example that can be checked by hand. We then read related literature (CRT and polynomial group keys, certificateless signatures, batch verification, mediated and threshold signatures, Byzantine fault tolerance) and designed fixes that use well-known tools in a new place.

6.2 Results at a glance

	Result	Chapter
F1	**TA impersonation:** the TA can produce a valid request for any UAV; key-escrow resilience and non-repudiation fail.	7
F2	**CRT group-key leakage:** a removed UAV reads new keys, a newcomer reads old keys, and a passive outsider who records two broadcasts recovers every key.	8
F3	**No forward secrecy:** one leaked long-term secret opens all past group keys.	9
C1	**Authentication with the TA offline** and a separately sealed key for every member.	10
C2	**Handover between TUAVs** with a one-time ticket and signatures in both directions.	11
C3	**Quorum eviction:** t TUAVs vote out a compromised TUAV.	12
C4	**Tether-split key:** a captured drone holds only half a key and cannot sign.	13
A1–A7	**Hardening:** seven flaws we found in our own first design and fixed.	14

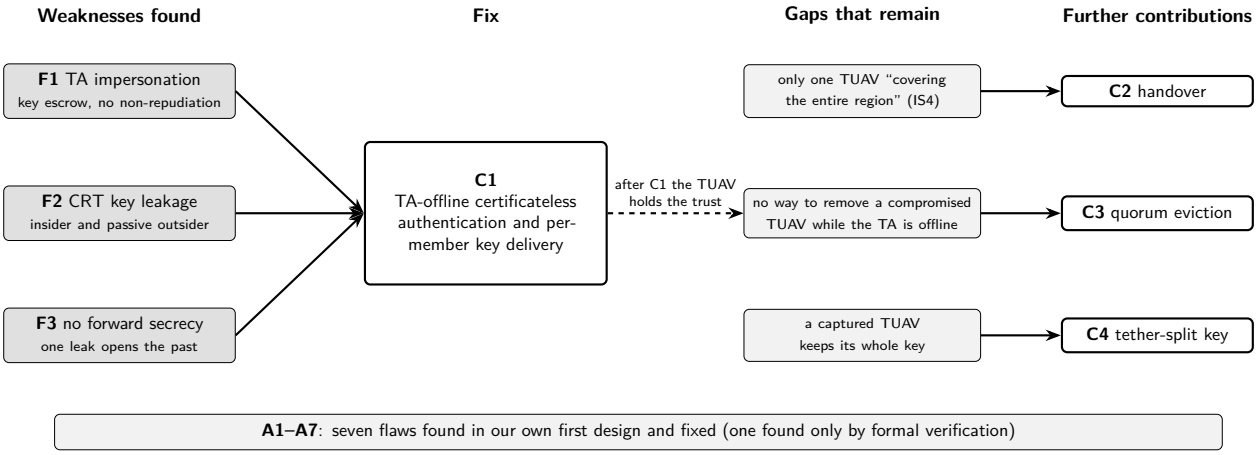


Figure 6.1: How our results fit together. Left: the three weaknesses we prove in the base scheme. They are all removed by contribution C1. After C1 the TA is offline, so the TUAV becomes the trusted party; this leaves three gaps (third column), which are closed by C2, C3 and C4. A1–A7 are improvements to our own design.

CHAPTER 7

F1: the TA can impersonate any UAV

Flaw F1 – key escrow: the TA can sign as any UAV

The TA stores every UAV's long-term secret s_i (IS2), and every other input of the credential Γ_i is public. Hence the TA – or anyone who steals the TA's database – can produce a request that GA1–GA4 accept in the name of *any* UAV, and nobody can tell it from a genuine one. Two properties ticked in the paper's Table II fail: **key-escrow resilience** and **non-repudiation**.

7.1 Who knows what

Look at the credential of Eq. (4.1), $\Gamma_i = [s_i\, h_3(ID_i, ts_2, s_i)\, R_{tu} - h_1(s_i\xi_i)]P - v_i\, h_1(r_i)\, Q_{tu}$:

Input of Γ_i	UAV	TA	Everyone	Source
s_i (long-term secret)	✓	✓	✗	IS2: issued and stored by the TA
r_i (random)	✓	✗	✗	US1
$h_1(r_i) = \xi_i$	✓	✓	✓	sent in the clear in US3
$ID_i,\ \upsilon_i,\ ts_2$	✓	✓	✓	sent in the clear in US3
$R_{tu},\ Q_{tu},\ ID_{tu}$	✓	✓	✓	TUAV beacon, IS5

The only value outside the TA's reach is r_i, and r_i enters Γ_i *only* through $h_1(r_i) = \xi_i$, which is public.

7.2 The theorem

Theorem 7.1 (TA forgery)**.** *Any party that knows $\langle id_i, s_i\rangle$ can, in polynomial time and without any value chosen by UAV i, produce a request that passes GA1–GA4 in the name of UAV i, and then obtain the group key through GD4–GD6.*

Proof. The forger runs Algorithm 7.1. Its output has exactly the form of an honest request in which the public value ξ' plays the role of $h_1(r_i)$. The correctness argument of Lemma 4.1 uses only the identity $h_1(r_i) = \xi_i$ and the definitions of υ_i, R_{tu} and Q_{tu} – never the value r_i itself – so it applies verbatim with $(\xi', ts', ID', \upsilon', \Gamma')$, and GA3 holds. In GD5 the forger computes $\mathsf{H}_1(ts_3, ID', h_1(s_i\xi')Q_{tu})$ from known values, and in GD6 it computes $\sigma' = h_1(s_i\xi')P$ and $\kappa_{tu} = \Lambda(\sigma') \bmod \sigma'$. Every step costs a constant number of hash evaluations and scalar multiplications. □

Algorithm 7.1: Forge(id_i, s_i) – impersonate UAV i

1. Read the current beacon $\langle ts_{tu}, ID_{tu}, R_{tu}, Q_{tu}\rangle$ (IS5).
2. Pick any $\xi' \in \mathbb{Z}_q^*$ and a fresh timestamp ts' – no r' is ever needed.
3. $ID' = h_3(id_i, ts', \xi'), \quad \upsilon' = h_4(ID', ID_{tu}, ts', \xi')$.
4. $\Gamma' = \big[s_i\, h_3(ID', ts', s_i)\, R_{tu} - h_1(s_i\xi')\big]P - \upsilon'\, \xi'\, Q_{tu}$.
5. Send $\langle \text{Request}, ts', ID', \xi', \upsilon', \Gamma'\rangle$ to the TUAV. GA1–GA4 accept.
6. On $\langle \text{ACK}, ts_3, ID', S', \{a_k\}\rangle$: $\sigma' = h_1(s_i\xi')P,\ \ \kappa_{tu} = \Lambda(\sigma') \bmod \sigma'$.

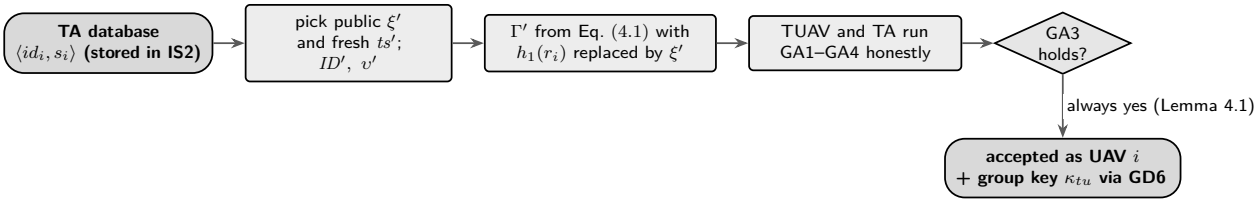


Figure 7.1: Flowchart of the TA-forgery attack (Algorithm 7.1). Starting from the stored pair $\langle id_i, s_i\rangle$, the forger builds a request with a public ξ'; the honest TUAV and TA then accept it. No step uses a value chosen by the real UAV.

7.3 Consequences

Corollary 7.2 (Key escrow and repudiation)**.** *(i) The base scheme is at Girault trust level 1: the authority can impersonate any user without leaving a trace. Key-escrow resilience (Table II, F4) does not hold. (ii) Non-repudiation (Table II, F10) does not hold: for any accepted request, the TA could have produced it. Moreover, only the TA can verify a request at all, since* $\mu_i = \hat{e}(h_3(ID_i, ts_2, s_i)P,\, s_iP)$ *needs* s_i*; a third party can never check a UAV's message.*

Corollary 7.3 (The TA reads every group key)**.** *In GA2 the TA computes every* σ_i*, so it can evaluate* $\kappa_{tu} = \Lambda(\sigma_i) \bmod \sigma_i$ *for every broadcast it observes.*

Remark 7.4 (The "certificateless" claim)*.* The proof of Theorem 4 states that "the certificateless authentication is adopted" with "r_i as the partial key". In a certificateless scheme the user's own secret must enter the *signing key*, so that the authority's partial key alone is useless [2]. Here r_i enters only as $h_1(r_i) = \xi_i$, and ξ_i is broadcast in US3. The user's "partial key" therefore contributes no secret, and the scheme is, in effect, a key-escrow scheme.

Remark 7.5 (Why the paper's Theorem 1 does not catch this)*.* The reduction of Theorem 1 lets an adversary $\mathcal{A}_1$ forge requests, but $\mathcal{A}_1$ is never given what the TA knows. A malicious or breached authority (a "Type II" adversary in certificateless terminology [2]) is outside the model, which is exactly the case of F1.

How C1 removes F1 (Chapter 10)

Each UAV adds its own secret x that never leaves the device: $sk = x + d$. The TA knows only the partial key d, so it cannot sign under the UAV's key (Theorem 10.6).

CHAPTER 8

F2: the CRT group key leaks – to members and to outsiders

Flaw F2 – the broadcast polynomial reveals the decrypting keys

The polynomial $\Lambda(x) = \kappa_{tu} \sum_i \omega_i \varphi_i + \prod_i (x - \sigma_i)$ hides only its *constant* term. Every other coefficient is a symmetric function of the members' decrypting keys, published in the clear.

- **F2(a) – revocation fails.** Any member recovers *every* σ_i; after being removed it still computes each new group key.
- **F2(b) – backward secrecy fails.** A UAV that joins recovers the old members' σ_i and opens recorded old group keys.
- **F2(c) – a passive outsider wins.** An eavesdropper who never registered records the two broadcasts around one join or leave and recovers every σ_i and *both* group keys.

8.1 One value is the same for everybody

Lemma 8.1 (Common value)**.** *Let* $C = \kappa_{tu} \sum_{i=1}^{n} \omega_i \varphi_i$*. Then* $\Lambda(\sigma_j) = C$ *for every member* j*.*

Proof. At $x = \sigma_j$ the product $\prod_i (x - \sigma_i)$ contains the factor $(\sigma_j - \sigma_j) = 0$, so only the constant C

remains. □

The paper uses this fact to deliver the key ($C \bmod \sigma_j = \kappa_{tu}$, Lemma 4.2). But the same fact means that every member knows the *mask* C itself, not only $C \bmod \sigma_j$.

8.2 The insider attack: F2(a) and F2(b)

Theorem 8.2 (Insider key recovery)**.** *A member j that knows only its own σ_j and the broadcast coefficients $\{a_k\}$ recovers every member's decrypting key in polynomial time.*

Proof. By Lemma 8.1 member j computes $C = \Lambda(\sigma_j)$, so it knows

$$\Lambda(x) - C = \prod_{i=1}^{n} (x - \sigma_i)$$

– the broadcast polynomial with a_0 replaced by $a_0 - C$. This is a monic integer polynomial whose roots are exactly the σ_i; all of them are integers (Remark 4.3), so by the Rational Root Theorem (2.5) and Proposition 2.6 they are found in time polynomial in n and the bit length. □

Corollary 8.3 (F2(a): revocation fails)**.** *Let member j be removed by DU1 and let k be any member that stays. From the broadcast before its removal, j knows σ_k (Theorem 8.2); DU2 keeps σ_k unchanged; hence j computes the new key $\kappa^*_{tu} = \Lambda^*(\sigma_k) \bmod \sigma_k$.*

Corollary 8.4 (F2(b): backward secrecy fails)**.** *Let UAV k join in DU1. Applying Theorem 8.2 to Λ^*, it learns the decrypting keys of the old members, which DU2 kept unchanged. For any recorded earlier broadcast Λ and any old member j, it computes the old key $\kappa_{tu} = \Lambda(\sigma_j) \bmod \sigma_j$.*

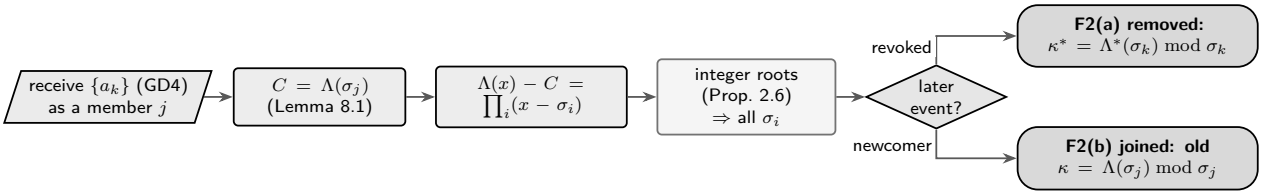


Figure 8.1: Flowchart of the insider attack (Theorem 8.2). A member evaluates the broadcast polynomial at its own key, subtracts the result and finds the roots; this gives every member's key. Depending on what happens later, the same member can break revocation, F2(a), or backward secrecy, F2(b).

8.3 The passive outsider attack: F2(c)

The insider attack needs one σ_j. The outsider attack needs none: it uses only the public coefficients of two broadcasts.

Lemma 8.5 (Vieta sum leak)**.** *If $n \geq 2$, the coefficient a_{n-1} of $\Lambda(x)$ equals $-(\sigma_1 + \cdots + \sigma_n)$. More generally a_k for $1 \leq k \leq n-1$ are the coefficients of $\prod_i (x - \sigma_i)$ exactly, without any mask.*

Proof. The mask C is a constant, so it changes only a_0. The remaining coefficients are those of $\prod_i (x - \sigma_i)$, given by Vieta's formulas (Theorem 2.4); in particular $a_{n-1} = -e_1(\sigma) = -\sum_i \sigma_i$. □

Theorem 8.6 (Passive outsider key recovery)**.** *Let Λ and Λ' be two consecutive broadcasts whose member sets differ by exactly one UAV with key σ, and suppose at least two members are common to both sets. An eavesdropper that knows only $\{a_k\}$ and $\{a'_k\}$ computes, in polynomial time, σ, every other decrypting key, the old group key κ_{tu} and the new group key κ^*_{tu}.*

Proof. Write $p(x) = \prod_{\text{old}}(x - \sigma_i) = \sum p_k x^k$ and $q(x) = \prod_{\text{new}}(x - \sigma_i) = \sum q_k x^k$. By Lemma 8.5 the eavesdropper knows every coefficient of p and q except the constants p_0 and q_0.

Leave ($\deg q = \deg p - 1$). Then $p(x) = (x - \sigma)\, q(x)$. The sums are public, so $\sigma = -p_{n-1} + q_{n-2}$. Comparing the coefficients of x^1 in $p = (x - \sigma)q$ gives $p_1 = q_0 - \sigma q_1$, hence

$$\boxed{q_0 = p_1 + \sigma\, q_1}$$

where $p_1 = a_1$ and $q_1 = a'_1$ are public (because at least two members remain, q_1 is not the masked constant). Now q is known completely, its roots are the remaining keys (Proposition 2.6), and $p = (x - \sigma)q$ is known too. With any key, the attacker evaluates $\Lambda(\sigma_j) \bmod \sigma_j$ and $\Lambda'(\sigma_j) \bmod \sigma_j$ like a member and obtains both group keys.

Join ($\deg q = \deg p + 1$). Now $q(x) = (x - \sigma)\, p(x)$, $\sigma = -q_n + p_{n-1}$, and the x^1 coefficients give $\boxed{p_0 = q_1 + \sigma\, p_1}$; the rest is identical with the roles of p and q swapped. □

Algorithm 8.1: OutsiderRecover($\{a_k\}, \{a'_k\}$) – two recorded broadcasts

1. $n \leftarrow \deg \Lambda$, $n' \leftarrow \deg \Lambda'$. If $|n - n'| \neq 1$, stop (outside the theorem).
2. Sums: $S = -a_{n-1}$, $S' = -a'_{n'-1}$; $\sigma \leftarrow |S - S'|$.
3. If $n' = n - 1$ (leave): $q_0 \leftarrow a_1 + \sigma\, a'_1$; $f(x) \leftarrow x^{n'} + a'_{n'-1}x^{n'-1} + \cdots + a'_1 x + q_0$ (the new product).
   If $n' = n + 1$ (join): $p_0 \leftarrow a'_1 + \sigma\, a_1$; $f(x) \leftarrow x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + p_0$ (the old product).
4. Find the integer roots of f (Proposition 2.6); together with σ these are all keys of both sets.
5. Output $\kappa_{tu} = \Lambda(\sigma_j) \bmod \sigma_j$ and $\kappa^*_{tu} = \Lambda'(\sigma_k) \bmod \sigma_k$ for any old j and new k.

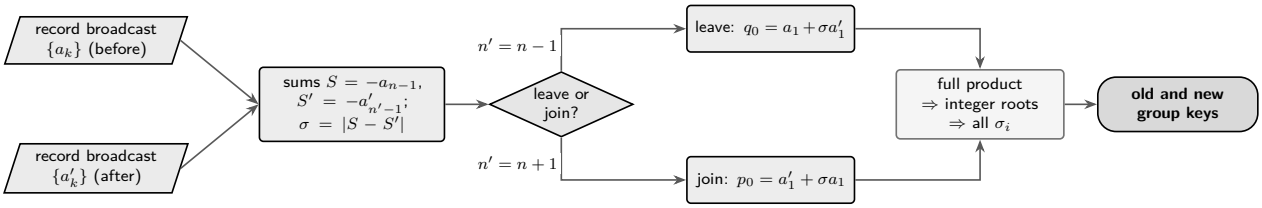


Figure 8.2: Flowchart of the passive outsider attack (Algorithm 8.1). The attacker records the coefficient sets before and after one membership change, obtains the changed member's key from the two coefficient sums, removes the mask, and finds all keys and both group keys. It needs no registration and no secret.

Several UAVs leaving (or joining) at once

DU1 is a *batch* update: $\ominus$ UAVs may leave or $\oplus$ may join in one step. The same algebra extends.

Proposition 8.7 (Batch changes)**.** *Suppose $m \geq 1$ UAVs leave in one update and $r \geq m + 1$ members remain (or m UAVs join a group of $r \geq m + 1$). Then the two broadcasts determine all keys of both groups in polynomial time.*

Proof. For a leave write $p = q \cdot d$ with $q = \prod_{\text{stay}}(x - \sigma_i)$ of degree r and $d = \prod_{\text{leave}}(x - \sigma_i) = x^m + d_{m-1}x^{m-1} + \cdots + d_0$. Comparing the coefficients of x^{n-j} for $j = 1, \ldots, m$ (with $n = r + m$) gives

$$d_{m-j} = p_{n-j} - \sum_{i=1}^{j} q_{r-i}\, d_{m-j+i},$$

which uses only p_{n-j} ($n - j \geq r \geq 1$, unmasked), q_{r-i} with $r - i \geq r - m \geq 1$ (unmasked) and already computed d's. So d is found top-down. The coefficient of x^m then gives $q_0 = p_m - \sum_{i=1}^{\min(m,r)} q_i\, d_{m-i}$. Now d and q are known; their roots are all keys. A batch join is the same with the roles of the two broadcasts swapped. □

Remark 8.8 (Scope). Theorem 8.6 is proved for one join or one leave between two broadcasts, and Proposition 8.7 for batches of pure leaves or pure joins with more members staying than changing. Two cases remain open: a *single* broadcast with no second one to compare, and one update that mixes joins and leaves. We also confirmed Theorem 8.6 numerically at the paper's own size (160-bit σ_i, 12 UAVs, one leave and one join): every σ_i and both group keys were recovered.

8.4 How this contradicts the paper

Quoted from the base paper

"the dynamic key batch updating process can be securely executed in the open wireless environment since the malicious entities that successfully collect the coefficient set $\{a_i\}_{i\in[0,n]}$ can not acquire the κ_{tu} without the correct decrypting key σ_i." (Section IV-D)

Theorem 8.6 refutes this sentence directly: an entity that collects two coefficient sets, and nothing else, acquires κ_{tu} *and* every σ_i. Corollaries 8.3 and 8.4 refute "supports multi-device revocation and new device participation" (Section IV-D) and the proof of Theorem 5 ("if the σ_i has not been removed by TUAV").

8.5 Root cause, and why a small patch is not enough

By Vieta's formulas the scheme *publishes* the elementary symmetric functions $e_1, \ldots, e_{n-1}$ of the members' secrets and masks only e_n. A set whose symmetric functions are public is not hidden: one member pins it down completely (Theorem 8.2), and two broadcasts that differ by one member reveal it to anyone (Theorem 8.6).

Two patches come to mind, and neither is attractive. (i) Giving every member a *fresh* σ_i at each update would stop the leaks across updates, but $\sigma_i = h_1(s_i\xi_i)P$ changes only with a new ξ_i, i.e. a new US/GA round through the TA for *every* member at *every* update – exactly the cost DU was meant to avoid. (ii) Masking all coefficients would require each member to remove the mask, i.e. a secret shared with each member – which is per-member key delivery. We therefore drop the public polynomial entirely (C1).

Remark 8.9 (Size of the broadcast). The coefficient a_k is a symmetric function of degree $n - k$ in 160-bit values, so it has about $160(n - k)$ bits, and one coefficient set has about $160(\frac{n(n-1)}{2} + n + 1)$ bits $\approx 10n^2$ bytes. GD3 sends it "one by one to each vehicle", so one key distribution moves about $10n^3$ bytes: roughly 17.4 MB for $n = 120$.

Limitations

All results use the integer reading of σ_i (Remark 4.3), which is the only reading under which GD6 is defined. CRT-based group-key schemes are already known to be fragile when a user is

corrupted [11]; our insider attack is an explicit instance of that weakness for this scheme. The passive *outsider* attack needs no corruption at all, and we have not found it reported for this scheme or its 94 citing works.

How C1 removes F2 (Chapter 10)

No function of the members' secrets is ever published. The group key travels as one AEAD ciphertext per member, each under its own fresh Diffie–Hellman key, and every join or leave triggers a fresh, independent key (Theorem 10.7).

CHAPTER 9

F3: no forward secrecy

Flaw F3 – one leaked long-term secret opens the past

The decrypting key $\sigma_i = h_1(s_i\xi_i)P$ is a deterministic function of the long-term secret s_i and of ξ_i, which is sent in the clear. Nothing ephemeral and secret enters it. So if s_i ever leaks – a captured UAV, a memory dump, a breached TA – an attacker who recorded the old traffic rebuilds the group key of **every past session** that UAV took part in. The paper's Table II ticks *forward secrecy* (F7).

9.1 The theorem

Theorem 9.1 (Retroactive key recovery)**.** *Let UAV i take part in sessions $1, \dots, T$, and let an eavesdropper record, for each session t, the request value $\xi_i^{(t)}$ (US3) and the coefficient set of $\Lambda^{(t)}$ (GD4). If s_i is revealed at any later time, the eavesdropper computes every past group key*

$$\sigma_i^{(t)} = h_1(s_i\,\xi_i^{(t)})P, \qquad \kappa^{(t)} = \Lambda^{(t)}(\sigma_i^{(t)}) \bmod \sigma_i^{(t)}, \qquad t = 1, \dots, T.$$

Proof. $\xi_i^{(t)}$ is public (US3) and s_i is now known, so $\sigma_i^{(t)}$ is computable; by Lemma 4.2 the GD6 computation then returns $\kappa^{(t)}$ exactly as it did for the UAV itself. No step uses a value that was erased after session t, because the scheme never creates one. □

Corollary 9.2 (One leak opens whole sessions)**.** *Combined with Theorem 8.2, the attacker also recovers every other member's $\sigma_j^{(t)}$ of those sessions and therefore every key update DU1–DU2 made while those members stayed.*

Corollary 9.3 (The TA can open the past at will)**.** *The TA stores every s_i (IS2), so it can apply Theorem 9.1 to any recorded session at any time.*

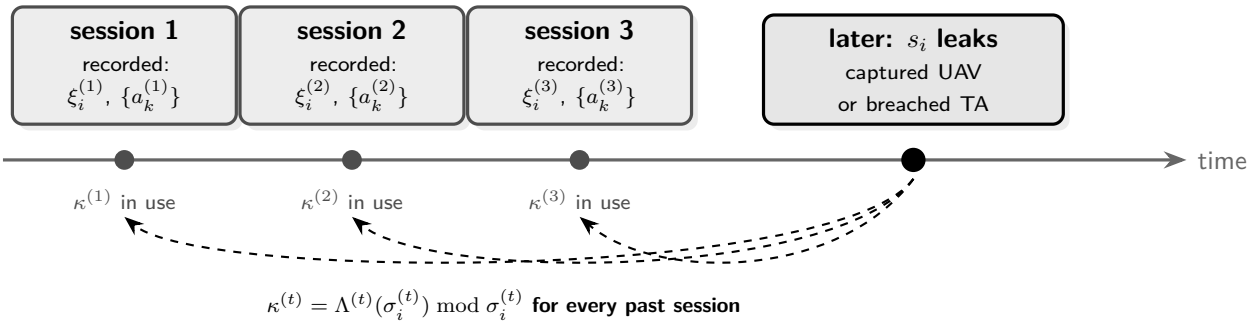


Figure 9.1: Timeline of the forward-secrecy failure. In each past session the attacker records the public values $\xi_i^{(t)}$ and $\{a_k^{(t)}\}$. When the long-term secret s_i leaks later, the attacker recomputes the group key of every recorded session (dashed arrows).

Remark 9.4 (Root cause). Forward secrecy needs a secret that exists only during the session and is then erased – typically an ephemeral Diffie–Hellman exponent. In the base scheme the only fresh value is r_i, and it enters every computation only as $h_1(r_i) = \xi_i$, which is broadcast. The authentication values $S_i = \mathsf{H}_1(ts_3, ID_i, h_1(s_i\xi_i)Q_{tu})$ of GD3–GD5 are likewise recomputable from s_i and public data.

How C1 removes F3 (Chapter 10)

Each key delivery uses fresh exponents e_a (TUAV) and e_i (UAV) that are erased afterwards. Recomputing an old pairwise key from long-term keys would require $e_a e_i P$ from $E_a = e_a P$ and $E_i = e_i P$, which is the CDH problem (Theorem 10.8).

CHAPTER 10

C1: TA-offline authentication with per-member key delivery

Key idea

The TA works only **before** the mission. Each entity holds a *certificateless* key $sk = x + d$: the TA issues the partial key d, the entity adds its own secret x that never leaves it. In the field the TUAV checks all join requests with the TA's public key alone, in one weighted batch, and sends each accepted UAV the group key *sealed under its own fresh Diffie–Hellman key.* No satellite, no pairing, no public polynomial.

10.1 Design goals

	Goal	Answers
G1	the authority that issues credentials cannot sign for anybody	F1
G2	nothing published depends on the members' secrets; removed members and newcomers learn nothing	F2
G3	every key delivery uses fresh, erased secrets	F3
G4	authentication needs only public parameters, so the TA can stay offline during the mission	design
G5	no pairings; many requests checked at once, and one bad request cannot block the others	design

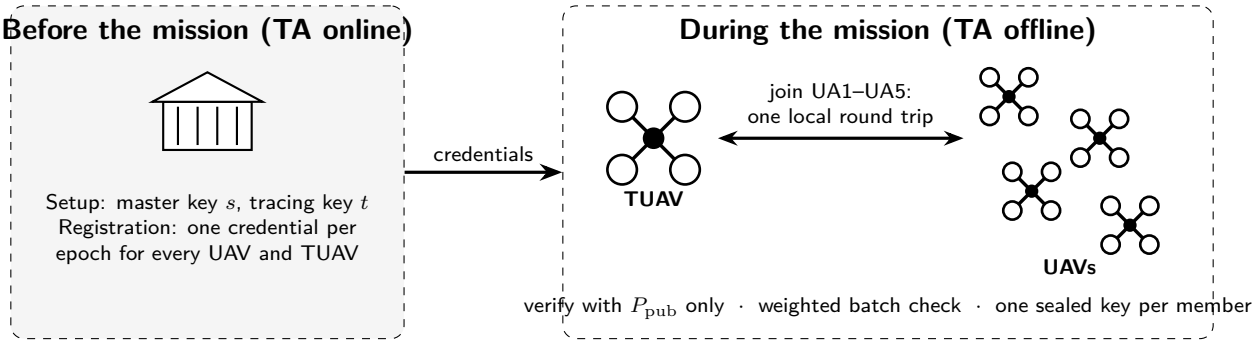


Figure 10.1: Overview of contribution C1. Left: before the mission, the TA sets up the system and issues one certificateless credential to every UAV and TUAV. Right: during the mission the TA is offline; a UAV joins its TUAV in one local exchange (steps UA1–UA5), the TUAV verifies requests using only the TA's public key P_{pub}, and each member receives the group key sealed under its own key.

10.2 Keys: certificateless, so the TA cannot sign

ST and RG1–RG4 (before the mission)

ST. The TA picks a master key s and a tracing key t, and publishes $P_{\text{pub}} = sP$ and $T_{\text{pub}} = tP$.
RG1. The entity picks its own secret x and sends $X = xP$ with a proof of possession. x **never leaves the entity.**
RG2. The TA picks ρ, y and computes a pseudonym and a partial key

$$PID = (PID^1, PID^2) = (\rho P,\ RID \oplus H_0(\rho T_{\text{pub}} \,\|\, ep)),$$
$$Y = yP, \qquad h = H_1(PID \,\|\, ep \,\|\, X \,\|\, Y), \qquad d = y + s\,h \bmod q,$$

where the label ep encodes the validity epoch and the role ($ep = 0$ is reserved for TUAVs).
RG3. The TA sends (PID, ep, Y, d) to the entity over the registration channel.
RG4. The entity checks $dP \stackrel{?}{=} Y + hP_{\text{pub}}$ and keeps $sk = x + d,\ pk = X + Y + hP_{\text{pub}}$.

Lemma 10.1 (Key consistency)**.** *$sk \cdot P = pk$, and anyone can compute pk from the public tuple (PID, ep, X, Y) and P_{pub}.*

Proof. $sk \cdot P = xP + yP + s\,hP = X + Y + hP_{\text{pub}} = pk$, and $h = H_1(PID \,\|\, ep \,\|\, X \,\|\, Y)$ is public. □

The pair (Y, d) is a Schnorr signature of the TA on (PID, ep, X); this is the Galindo–Garcia identity-based construction [3], made certificateless by adding the user's own x [2]. Only the tracing authority, which knows t, can compute $\rho T_{\text{pub}} = t\,PID^1$ and open PID^2; a fresh ρ per epoch makes pseudonyms of different epochs unlinkable.

10.3 Joining a TUAV: UA1–UA5

UA1–UA5 (in the field, TA offline)

UA1. TUAV $\mathcal{T}_a$ picks e_a, sets $E_a = e_aP$ and broadcasts a beacon $\langle ID_a, X_a, Y_a, E_a, ts_a \rangle$ signed with its own key.

UA2. UAV $\mathcal{U}_i$ verifies the beacon, picks e_i, r_i and sends $\langle PID_i, ep, X_i, Y_i, E_i, ts_i, R_i, v_i\rangle$, where

$$E_i = e_i P, \qquad R_i = r_i P, \qquad m_i = (\texttt{join} \,\|\, ID_a \,\|\, E_a \,\|\, E_i),$$
$$c_i = H_2(m_i \,\|\, PID_i \,\|\, ep \,\|\, X_i \,\|\, Y_i \,\|\, R_i \,\|\, ts_i), \qquad v_i = r_i + c_i\, sk_i.$$

UA3. The TUAV draws random 64-bit weights λ_i and checks all n requests at once:

$$\Big(\sum_i \lambda_i v_i\Big) P \stackrel{?}{=} \sum_i \lambda_i R_i + \sum_i (\lambda_i c_i)(X_i + Y_i) + \Big(\sum_i \lambda_i c_i h_i\Big) P_{\text{pub}}. \tag{10.1}$$

If the check fails, it verifies each request alone, accepts the valid ones and lists the invalid pseudonyms (bounded fallback).

UA4. For every accepted $\mathcal{U}_i$: $K_i = \mathsf{KDF}(e_a E_i \,\|\, PID_i \,\|\, ID_a \,\|\, ts_i)$ and $C_i = \mathsf{Enc}(K_i, \; GK \,\|\, \nu; \; H_3(PID_i) \,\|\, \nu \,\|\, ts_3)$. All entries go out in **one** broadcast $\langle ID_a, ts_3, \nu, \{H_3(PID_i), C_i\}_i\rangle$.

UA5. $\mathcal{U}_i$ computes $K_i = \mathsf{KDF}(e_i E_a \,\|\, PID_i \,\|\, ID_a \,\|\, ts_i)$, opens C_i (which also proves the TUAV knows e_a), stores (GK, ν) and **erases** e_i. The TUAV erases e_a after the round.

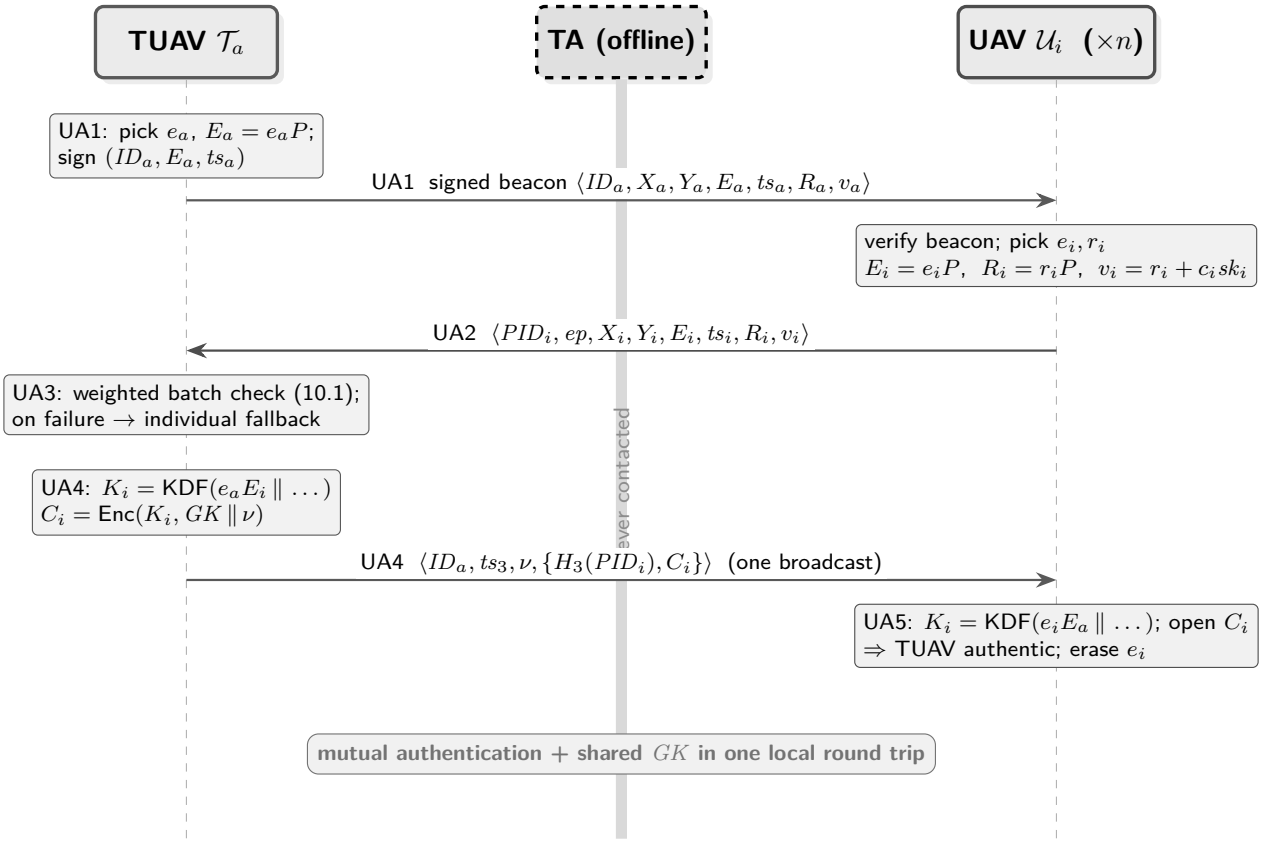


Figure 10.2: Message flow of C1 between the TUAV and n UAVs. Time runs from top to bottom; boxes show local computation. The TA (centre) is never contacted. Compared with Figure 4.3, there is no satellite link, no pairing and no CRT polynomial.

10.4 Rekeying on every change

RK1–RK2 (join and leave)

RK1 (leave or eviction). The TUAV draws a *fresh* GK', sets $\nu \leftarrow \nu + 1$ and sends $C_i' = \mathsf{Enc}(K_i, GK' \,\|\, \nu)$ only to the members that stay.
RK2 (join). After admitting newcomers (UA1–UA5), the TUAV immediately rekeys the whole cell, so a newcomer never receives a key that was already in use (fix A6). Members accept only increasing versions ν (fix A5).

Remark 10.2 (Large groups). Each rekey costs one entry per member. For large cells the TUAV can arrange keys in a logical key hierarchy (LKH) [13], which reduces a rekey to $O(\log n)$ entries; nothing else changes.

10.5 Correctness

Lemma 10.3 (Signature and batch correctness)**.** *(i) An honest request satisfies $v_i P = R_i + c_i pk_i$. (ii) A batch of honest requests satisfies Eq.* (10.1) *for every choice of weights. (iii) Both sides of UA4/UA5 derive the same K_i.*

Proof. (i) $v_i P = r_i P + c_i sk_i P = R_i + c_i pk_i$ by Lemma 10.1. (ii) Multiply (i) by λ_i, sum over i, and expand $pk_i = X_i + Y_i + h_i P_{\text{pub}}$. (iii) $e_a E_i = e_a e_i P = e_i E_a$, and the other KDF inputs are identical. □

10.6 Security

Theorem 10.4 (Unforgeability)**.** *In the random-oracle model, if the ECDLP is hard in $\mathbb{G}$, no PPT adversary can output a valid signature on a new message under the credential (PID_i, ep, X_i, Y_i) of an honest entity – neither* ***(Type I)*** *an outsider who does not know s but may register its own keys and replace other public keys, nor* ***(Type II)*** *a malicious TA who knows s and every partial key d_i.*

Proof sketch. Type II. Given an ECDLP challenge X^*, the reduction plays the TA honestly (it knows s and chooses y) and sets $X_i = X^*$ for one guessed honest entity. It answers signing queries without sk_i by the standard Schnorr simulation: pick v, c, set $R = vP - c\,pk_i$ and program H_2. A forgery is rewound with the General Forking Lemma (2.8) on H_2, which yields $sk_i = x_i + d_i$; since d_i is known, $x_i = sk_i - d_i$ solves the challenge.
Type I. If the forgery is under the honest (X_i, Y_i), the same argument applies with the challenge planted in X_i. If the adversary uses keys (X', Y') of its own, the proof of possession of X' gives the reduction x', and forking on H_2 yields $sk' = x' + d'$, hence d' with $d'P = Y' + h' P_{\text{pub}}$. Because $h' = H_1(PID \,\|\, ep \,\|\, X' \,\|\, Y')$ is fixed only after Y', a second fork on H_1 gives d'' with $d''P = Y' + h'' P_{\text{pub}}$, and $s = (d' - d'')(h' - h'')^{-1}$ solves an ECDLP challenge planted in P_{pub}. This double-forking structure is that of the Galindo–Garcia scheme [3]; a careful analysis of its (non-tight) reduction is given in [4]. □

Theorem 10.5 (Batch soundness)**.** *If a batch contains at least one invalid request, Eq.* (10.1) *holds with probability at most 2^{-64} over the TUAV's weights. With the fallback, the TUAV accepts exactly the valid requests, so one forged request cannot block the honest ones.*

Proof. Eq. (10.1) is $\sum_i \lambda_i \Delta_i = 0$ with $\Delta_i = v_i P - R_i - c_i pk_i$; an invalid request has $\Delta_i \neq 0$. Apply the Small-Exponent Test (Lemma 2.9) with $\ell = 64$. The fallback checks each request individually, which is exact. □

Theorem 10.6 (No key escrow)**.** *The TA cannot produce a valid signature under an honest entity's credential (X_i, Y_i) unless it solves the ECDLP. It can only issue a* second *credential (X', Y') for the same pseudonym – Girault trust level 2, compared with level 1 in the base scheme (Corollary 7.2).*

Proof. This is the Type II case of Theorem 10.4: the TA knows d_i but not x_i, and $sk_i = x_i + d_i$. □

Theorem 10.7 (Group-key secrecy, revocation and backward secrecy)**.** *Assume CDH is hard in $\mathbb{G}$, KDF is a random oracle, Enc is a secure AEAD scheme and signatures are unforgeable (Theorem 10.4). Then the group key GK_ν of version ν is indistinguishable from random for (a) any active outsider, (b) any member removed before version ν, and (c) any member that joined after version ν.*

Proof sketch. GK_ν appears only inside the ciphertexts $C_i = \mathsf{Enc}(K_i, GK_\nu \,\|\, \nu)$ of the members of version ν. Each K_i is derived from $e_a E_i$, where E_a is signed by the TUAV (UA1) and E_i is signed by the member inside m_i (UA2), so an active adversary cannot substitute its own value without forging a signature. Computing $e_a E_i$ from E_a and E_i is CDH; in the random-oracle model K_i is therefore uniform to everyone except the two endpoints, and AEAD security makes C_i independent of GK_ν. (a) An outsider holds no K_i. (b) A removed member gets no entry for version ν (RK1); its old keys open only older entries, and GK_ν is drawn fresh and independently. (c) A newcomer is rekeyed on arrival (RK2), so it receives a fresh key independent of the earlier ones, which were sealed under other members' keys. Unlike the base scheme, no published value depends on any member's secret, so Theorems 8.2 and 8.6 have nothing to work on. □

Theorem 10.8 (Forward secrecy)**.** *If every long-term secret (s, all x_i and d_i, the TUAV's key) is revealed after version ν has ended and the exponents e_a, e_i have been erased, GK_ν remains indistinguishable from random under CDH.*

Proof. Long-term keys are used only to sign; they do not enter K_i. Recovering K_i requires $e_a e_i P$ from the recorded $E_a = e_a P$ and $E_i = e_i P$, which is a CDH instance; the rest follows as in Theorem 10.7. □

10.7 How C1 answers the three flaws

Flaw	What C1 does, mathematically	Result
F1 TA impersonation	$sk = x + d$; the TA never sees x	Thm. 10.6
F2 CRT leakage	no public function of members' secrets; one sealed entry per member; fresh key on every join and leave	Thm. 10.7
F3 no forward secrecy	keys come from ephemeral e_a, e_i, erased after use	Thm. 10.8

10.8 Cost, computed with the base paper's own constants

The base paper (Section VI-A) prices its operations with the MIRACL library: pairing $T_P = 7.2232$ ms, scalar multiplication in $\mathbb{G}_1$ $T_{SM \cdot P} = 2.6784$ ms, point addition in $\mathbb{G}_1$ $T_{PA \cdot P} = 0.0342$ ms, scalar

multiplication on the ordinary curve $T_{SM\cdot E} = 0.9018\,\text{ms}$, small-scalar multiplication $T_{SSM\cdot E} = 0.0475\,\text{ms}$, point addition $T_{PA\cdot E} = 0.0107\,\text{ms}$ and hashing $T_{SH} = 0.0012\,\text{ms}$. C1 needs only the ordinary-curve operations.

Authenticating and keying n UAVs	**Base scheme**	**C1**
Pairings	$3n$ (at the TA, GA2)	**0**
Satellite round trips	1 per login round	**0**
TA work per UAV	$3T_P + 3T_{SM\cdot P} + T_{PA\cdot P} \approx 29.74\,\text{ms}$	**0** (offline)
TUAV work	$1.2524n + 0.9054\,\text{ms}$ (the paper's own GA formula)	$(2n{+}4)T_{SM\cdot E} + nT_{SSM\cdot E} + 3nT_{PA\cdot E} + 4nT_{SH} \approx 1.888n + 3.607\,\text{ms}$
Total for $n = 120$	$\approx 3720\,\text{ms}$ + satellite trip	$\approx$ **230 ms**
UAV work	1.82 ms for US (as reported) + GD5 + GD6	$6T_{SM\cdot E} + 3T_{PA\cdot E} + 5T_{SH} \approx 5.45\,\text{ms}$
Key-distribution traffic	$\approx 10n^3$ bytes (Remark 8.9)	$\approx 80n$ bytes (one broadcast)

The UAV in C1 does more work than the paper's US phase because it also authenticates the TUAV and runs a Diffie–Hellman exchange; that is the price of mutual authentication and forward secrecy, and it stays a few milliseconds. On a modern 128-bit curve all scalar multiplications become somewhat more expensive, but C1 still needs no pairing and no satellite.

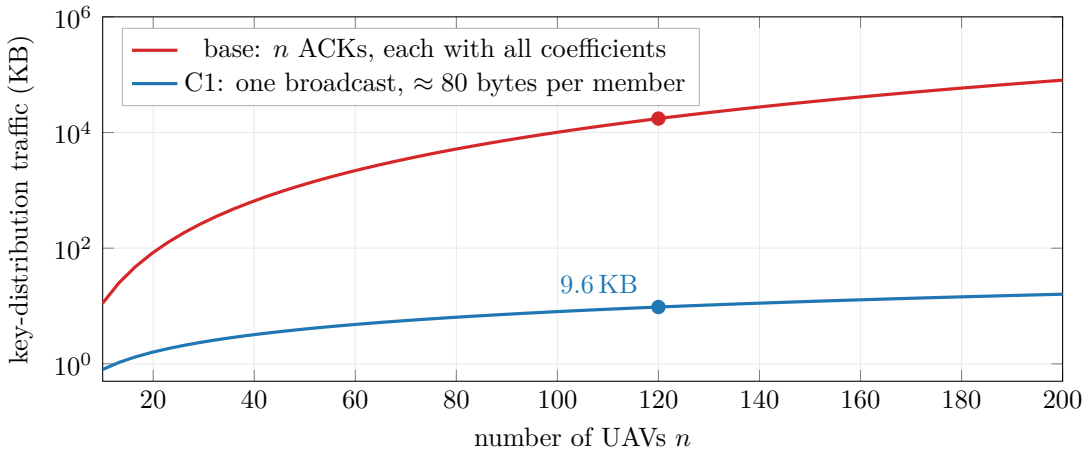


Figure 10.3: Traffic needed to distribute one group key against the number of UAVs n (logarithmic vertical axis), computed from the message formats. The base scheme sends all polynomial coefficients to each UAV, about $10n^3$ bytes in total; C1 sends one broadcast with about 80 bytes per member. The marked points are the values for $n = 120$.

Limitations

- The TA can still enrol a *phantom* member under a new pseudonym; C1 removes its power to sign as an *existing* entity (level 2, not 3).
- The security reductions use the forking lemma and are not tight [4]; this is standard for Schnorr-type schemes.
- After C1 the TUAV knows every K_i and the group key: **the trust has moved from the TA to the TUAV**. C2–C4 protect exactly this point.

CHAPTER 11

C2: handover between TUAVs

Key idea

A disaster zone needs several TUAVs, and UAVs fly between their areas ("cells"). Neighbouring TUAVs share a link key K_{ab}. The old TUAV gives a moving member a sealed, one-time **ticket**; the new TUAV admits it after checking the ticket, **the member's signature** and one fresh Diffie–Hellman exchange, and replies with **its own signature**. No TA, no satellite, no full re-login.

11.1 The gap in the base paper

Quoted from the base paper

IS4: "the TUAV, with the effective range covering the entire region, will periodically generate the random number r_{tu} ..."

The scheme assumes a single TUAV for the whole region. A tethered drone covers a few kilometres at most, so a real deployment has several TUAVs, and the paper has no TUAV-to-TUAV link, no handover and no leave message. A UAV that crosses into another cell can only start again: US1–US3, GA1 to the TA over the satellite (three pairings at the TA), and a rebuilt $\Lambda^*(x)$ at the new TUAV – while the old TUAV is never told that the UAV left.

11.2 Two ways to move

Path 1 – full join (always available). Any TUAV can verify any credential with P_{pub} alone (C1), so a UAV can simply run UA1–UA5 with the new TUAV. The new TUAV rekeys on arrival and the old TUAV rekeys after the departure.
Path 2 – ticket handover (C2). For a planned move, the member carries a ticket from the old TUAV and does not wait for the new TUAV's next beacon. The ticket idea goes back to Kerberos [18]; what is new here is binding it to signatures in both directions so that a *captured* TUAV cannot abuse it.

11.3 The protocol HO0–HO4

HO0–HO4 Inter-TUAV link and ticket handover

HO0. Neighbouring TUAVs exchange signed ephemeral values and derive $K_{ab} = \mathsf{KDF}(e_{ab}E_{ba} \,\|\, ID_a \,\|\, ID_b \,\|\, ts)$.
HO1. Before member $\mathcal{M}$ (pairwise key $K_{\mathcal{M}}$) leaves, $\mathcal{T}_a$ derives a handover key $hk = \mathsf{KDF}(K_{\mathcal{M}} \,\|\, \texttt{HO} \,\|\, ID_b \,\|\, ts_h)$ and seals

$$Tk = \mathsf{Enc}\big(K_{ab},\ PID \,\|\, ep \,\|\, pk \,\|\, hk \,\|\, N \,\|\, exp;\ ID_a \,\|\, ID_b\big)$$

with a fresh nonce N and an expiry time exp; it sends $\mathsf{Enc}(K_{\mathcal{M}}, Tk)$ to $\mathcal{M}$.
HO2. In the new cell $\mathcal{M}$ derives hk itself, picks e, sets $E = eP$ and sends Tk, E, $\mathsf{MAC}(hk, Tk \,\|\, E \,\|\, ts)$ and **its own signature** on $(ID_a \,\|\, ID_b \,\|\, H(Tk) \,\|\, E)$.
HO3. $\mathcal{T}_b$ opens Tk with K_{ab} and rejects if it has expired, if N was used before, if $\mathcal{M}$ has

been evicted, if the MAC fails, or if the signature does not verify under the pk inside the ticket. It picks e_b, derives $K' = \mathsf{KDF}(e_bE \,\|\, hk \,\|\, PID \,\|\, ID_b)$, rekeys its cell, and replies with E_b, $\mathsf{Enc}(K', GK_b \,\|\, \nu)$ and **its own signature** on $(ID_a \,\|\, ID_b \,\|\, H(Tk) \,\|\, E \,\|\, E_b)$.

HO4. $\mathcal{M}$ checks $\mathcal{T}_b$'s signature, derives $K' = \mathsf{KDF}(eE_b \,\|\, hk \,\|\, PID \,\|\, ID_b)$ and opens GK_b. $\mathcal{T}_a$ rekeys once $\mathcal{M}$ has left.

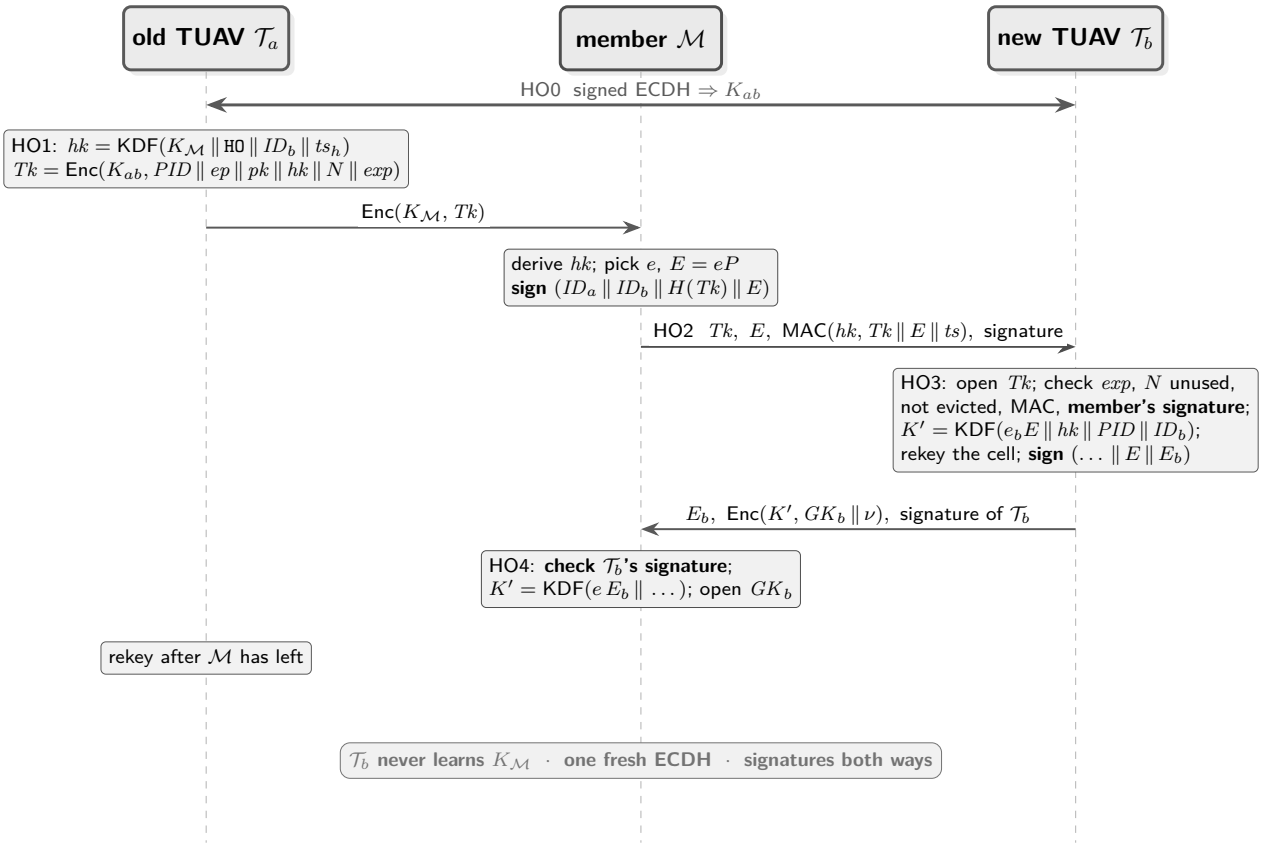


Figure 11.1: Message flow of the C2 handover between the old TUAV $\mathcal{T}_a$, the moving member $\mathcal{M}$ and the new TUAV $\mathcal{T}_b$. Time runs from top to bottom. The two signatures written in bold (by the member in HO2 and by the new TUAV in HO3) are the fixes A2 and A7 of Chapter 14.

11.4 Why each part is there

- hk **instead of** $K_{\mathcal{M}}$**:** the new TUAV never learns the member's old key, so a later compromise of $\mathcal{T}_b$ does not expose the old cell.
- **Fresh ECDH:** K' depends on e_bE, so learning K_{ab} or hk later does not reveal it (forward secrecy).
- **Nonce, expiry and both names:** a ticket works once, only before *exp*, and only between these two TUAVs.
- **The member's signature (A2):** without it, a captured old TUAV could mint tickets on its own; now a ticket alone admits nobody.
- **The new TUAV's signature (A7):** a captured old TUAV knows hk and could otherwise pose as the new TUAV; now it cannot.
- **Rekey on arrival and after departure:** the newcomer cannot read the new cell's past; the leaver cannot read the old cell's future.

11.5 Security

We modelled HO0–HO4 in the applied pi calculus and checked it with the ProVerif verifier [27] against a Dolev–Yao attacker [26] who also controls a *captured old TUAV* (it knows K_{ab}, $K_{\mathcal{M}}$ and hk).

Model (old TUAV captured)	Result of formal verification
Ticket alone admits the member	✗ attack: a member is "admitted" that never started a handover, and the new cell's key leaks $\Rightarrow$ **A2**
Member signs, the new TUAV's reply is unsigned	✗ attack: the member completes a handover with the captured TUAV posing as $\mathcal{T}_b$ $\Rightarrow$ **A7**
Both signatures (final protocol)	✓ all five properties hold

Theorem 11.1 (Handover security, symbolic model)**.** *In the Dolev–Yao model with a compromised old TUAV, the final protocol HO0–HO4 satisfies: (1) injective agreement of $\mathcal{T}_b$ with $\mathcal{M}$ (every accepted handover was started by $\mathcal{M}$, once); (2) injective agreement of $\mathcal{M}$ with $\mathcal{T}_b$; (3) secrecy of K'; (4) secrecy of the new cell key GK_b; and (5) forward secrecy of K' when K_{ab} and hk leak after the session.*

Justification. Established by automatic verification of the model above. Intuitively: (1) needs the member's signature, which the captured TUAV cannot produce (Theorem 10.4); (2) needs $\mathcal{T}_b$'s signature on E and E_b; (3)–(5) follow because K' contains e_bE, a CDH value unknown to the attacker even when it holds K_{ab} and hk. □

11.6 Cost and latency

In computation a ticket handover costs about the same as a full join: a few scalar multiplications on each side (one Diffie–Hellman exchange, one signature and one verification each) plus the cell rekey. Its gain is *latency*: in a full join the member must wait for the new TUAV's next signed beacon, on average half a beacon period (500 ms for a 1 s period), whereas with a ticket it starts HO2 at once. On the air, a crossing adds one member signature (65 bytes) and one signed reply (about 131 bytes). No message ever goes to the TA.

Limitations

- A ticket must never be accepted from a TUAV that has been evicted (Chapter 12): its tickets are sealed under keys a compromised TUAV holds. Members leaving an evicted TUAV use the full join.
- C2 protects the handover itself; a compromised *new* TUAV is handled by C3 and C4, not by C2.

CHAPTER 12

C3: evicting a compromised TUAV by quorum

Key idea

With the TA offline, nobody in the field can revoke a TUAV whose software has been taken over. In C3, t **neighbouring TUAVs vote**: each signs "evict T_{target}", and t distinct votes form an **eviction certificate** that every UAV and TUAV can check with public keys alone. Members of the evicted TUAV stop trusting it and re-join a neighbour by the full join.

12.1 Why it is needed

After C1 the TUAV is the root of trust of its cell: it admits members and holds every pairwise key and the group key. A TUAV can be lost in two ways. **Physical capture** – the drone leaves the tether and falls into the wrong hands – is handled by C4 (Chapter 13). **Software compromise while still tethered** – a hijacked radio stack or a malicious update – cannot be noticed by the tether, and the base paper has no revocation mechanism for TUAVs at all. C3 covers this case without the TA.

12.2 Votes and certificates

TUAV j votes with its ordinary C1 key (role label $ep = 0$):

$$vote_j = \text{Sign}_{sk_j}(\texttt{evict-tuav} \,\|\, ID_{\text{target}} \,\|\, ts_j) = (R_j, v_j).$$

Definition 12.1 (Valid eviction certificate)**.** A set of votes $\mathcal{C}$ is a valid certificate against T_{target} when all four hold:

V1. it contains votes from at least t *distinct* TUAV identities;

V2. T_{target} is not among the signers;

V3. every vote names the same target;

V4. the weighted batch check (10.1) holds over the votes, with $h_j = H_1(ID_j \,\|\, 0 \,\|\, X_j \,\|\, Y_j)$.

Because TUAV credentials are bound to $ep = 0$ inside h_j, a member's credential can never cast a vote (role binding, fix A4).

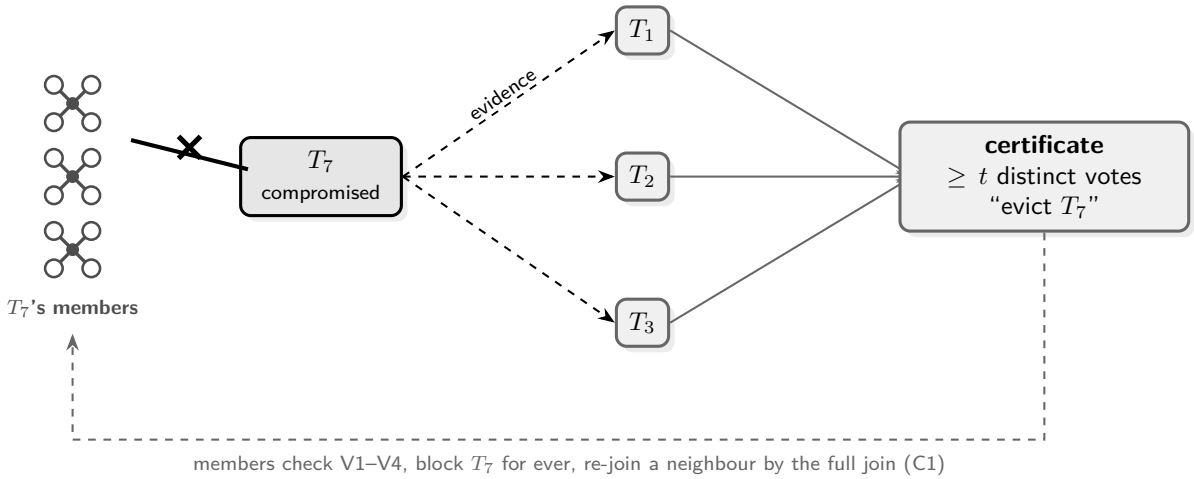


Figure 12.1: Quorum eviction (C3). The neighbours T_1, T_2, T_3 obtain evidence against the compromised TUAV T_7 (dashed arrows) and each signs a vote. At least t distinct votes form an eviction certificate, which is delivered to the members of T_7; they verify it and re-join a neighbour.

12.3 The protocol

EV1–EV5 Eviction with the TA offline

EV1 (evidence). A TUAV obtains evidence against a neighbour: *cryptographic* evidence – two conflicting messages signed by the target (for instance two different rekeys with the same version ν), or a signed message that breaks a rule (a decreasing version, a ticket for an evicted member) – or *operational* evidence such as a crew or sensor report.
EV2 (vote). It signs $vote_j$ and sends it, with the evidence, to its neighbours.
EV3 (assemble). Any TUAV that holds t distinct valid votes forms the certificate $\mathcal{C}$.
EV4 (spread). $\mathcal{C}$ is broadcast in the neighbouring cells and relayed into the target's cell.
EV5 (act). A member checks V1–V4, blocks the target *for ever*, discards its keys, and re-joins a neighbouring TUAV by the full join UA1–UA5 – never with a C2 ticket from the evicted TUAV. Neighbouring TUAVs refuse its tickets and link offers.

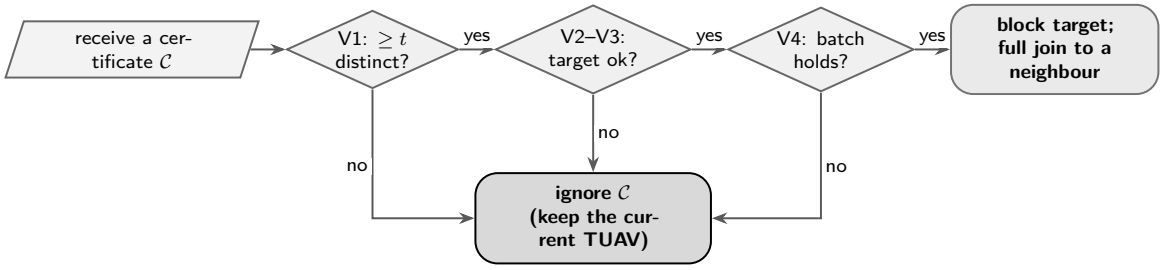


Figure 12.2: Flowchart of how a member checks an eviction certificate against the four conditions V1–V4 of Definition 12.1. Only if all conditions hold does the member leave its TUAV.

12.4 Security

Theorem 12.2 (Eviction safety)**.** *Suppose at most $f \leq t-1$ TUAVs are compromised, signatures are unforgeable (Theorem 10.4), and honest TUAVs vote only on evidence of misbehaviour. Then, except with negligible probability, no valid certificate names an honest TUAV whenever the evidence*

is cryptographic; for operational evidence the same holds as long as no honest voter is misled.

Proof. A valid certificate carries t valid votes from distinct identities (V1, V4). At most $f < t$ of them can come from compromised TUAVs, and producing a valid vote under an honest TUAV's key is a forgery. Hence at least one honest TUAV voted, which it does only with evidence against the target. Cryptographic evidence against an honest TUAV would consist of messages carrying its valid signatures that it never produced – again a forgery. □

Theorem 12.3 (Eviction liveness)**.** *If at least t honest TUAVs obtain evidence against a TUAV and can reach its members, a valid certificate forms and every member that receives it stops using that TUAV.*

Proof. The t honest votes name the same target and come from distinct identities other than the target, so V1–V3 hold; V4 holds by Lemma 10.3(ii). EV5 then applies. □

Corollary 12.4 (Choosing t)**.** *If up to f of a TUAV's n_T neighbours may be compromised, choose $t = f + 1$ for safety; liveness then needs $n_T - f \geq t$, i.e. $n_T \geq 2f + 1$.*

Remark 12.5 (Relation to PBFT). The pattern – a quorum of signed statements that anyone can check – is the view change of PBFT [19], where a new leader must show $2f+1$ signed messages out of $3f + 1$ replicas. PBFT needs the larger quorum because replicas must agree on an *order* of requests. Eviction is a monotone flag ("evicted" never reverts) with no order to agree on, so a quorum of $f + 1$ already prevents framing. C3 is a new *use* of this pattern – removing a key-management node with the authority offline – not new cryptography.

12.5 Cost

A vote is one signature. A member checks t votes in one weighted batch, about $t + 2$ scalar multiplications; for $t = 2$ that is $4 \times 0.9018 \approx 3.6$ ms with the base paper's constants.

Limitations

- t or more compromised TUAVs can still evict honest ones. C4 reduces this risk: a *captured* drone cannot sign, so it cannot vote.
- C3 decides *whether* to evict from evidence; it does not detect a compromise by itself.
- Members that cannot reach any honest neighbour cannot re-join until one comes into range.

CHAPTER 13

C4: the tether-split key

Key idea

The TUAV's signing key is split in two, $sk = a + g$. **The drone holds a; a small secure chip inside the ECRV holds** g, and the two are joined by the tether. The drone can sign only when the chip adds its half over the wire. **If the tether is cut, the drone's half is useless**, and the ECRV broadcasts a pre-signed *kill notice*. Verifiers notice nothing: the public key and the signature are exactly as before.

13.1 Why we add it: the TUAV is the new weak spot

After C1 the TA has left the field, so the TUAV holds all the trust in its cell – and it is the drone most worth capturing. The base paper never asks what happens then.

Design	What a thief obtains by capturing the TUAV
Base paper	s_{tu}, κ_{tu} and every member's σ_i; beacons need only s_{tu} (IS4) and key updates (DU1–DU2) are not signed, so the thief can quietly keep control of the group key
C1 alone	the whole key sk_a: it can run a fake TUAV from a laptop and a radio, signing beacons, handover replies and C3 votes
C1 + C4	**half a key, which signs nothing**, plus the current group key until the next rekey

While the drone is on the tether it hovers above the crew; it can fall into a stranger's hands only *after* it leaves the tether – a cut cable, a power loss, an interceptor, or the ECRV driving away. C4 ties the key to exactly that moment.

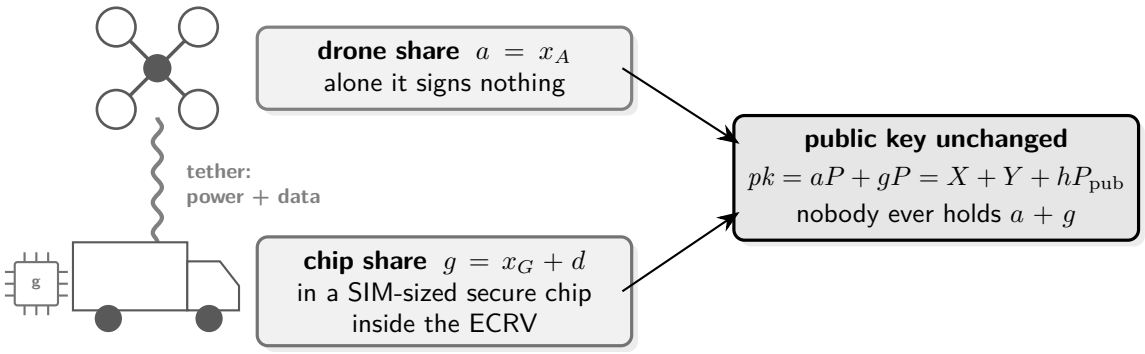


Figure 13.1: The tether-split key (C4). The drone holds the share a and a secure chip in the ECRV holds the share g; the two are connected by the tether. The public key $pk = aP + gP$ is unchanged, and no single device holds the full key $a + g$.

13.2 The construction

S1–S4 Setup at the depot (once, before the mission)

S1. The drone picks x_A and publishes $X_A = x_A P$; the chip picks x_G and publishes $X_G = x_G P$ (each with a proof of possession); $X = X_A + X_G$.
S2. The TA registers the TUAV as in RG2 with $ep = 0$: $Y = yP$, $h = H_1(ID_a \,\|\, 0 \,\|\, X \,\|\, Y)$, $d = y + s\,h$, and gives d **to the chip only**.
S3. The chip keeps $g = x_G + d$; the drone keeps $a = x_A$.
S4. Together they sign a **kill notice** $KN =$ "ID_a is out of service" and store it in the ECRV.

J1–J5 Joint signing of a message m over the tether (commit-then-reveal)

J1. Drone: pick r_A, $R_A = r_A P$; send m, ts and the *commitment* $C_A = H(R_A)$ to the chip.
J2. Chip: refuse unless the tether is alive, m is an allowed type (beacon, link offer, handover reply, C3 vote), ts is fresh and increasing, and no other session is open. Pick r_G and send $R_G = r_G P$.
J3. Drone: send R_A. Chip: check $H(R_A) = C_A$.
J4. Both: $R = R_A + R_G$, $c = H_2(m \,\|\, ID_a \,\|\, 0 \,\|\, X \,\|\, Y \,\|\, R \,\|\, ts)$. Chip sends $v_G = r_G + c\,g$ and logs (m, ts).
J5. Drone: $v_A = r_A + c\,a$ and $v = v_A + v_G$. The signature (R, v) has the usual form and size.

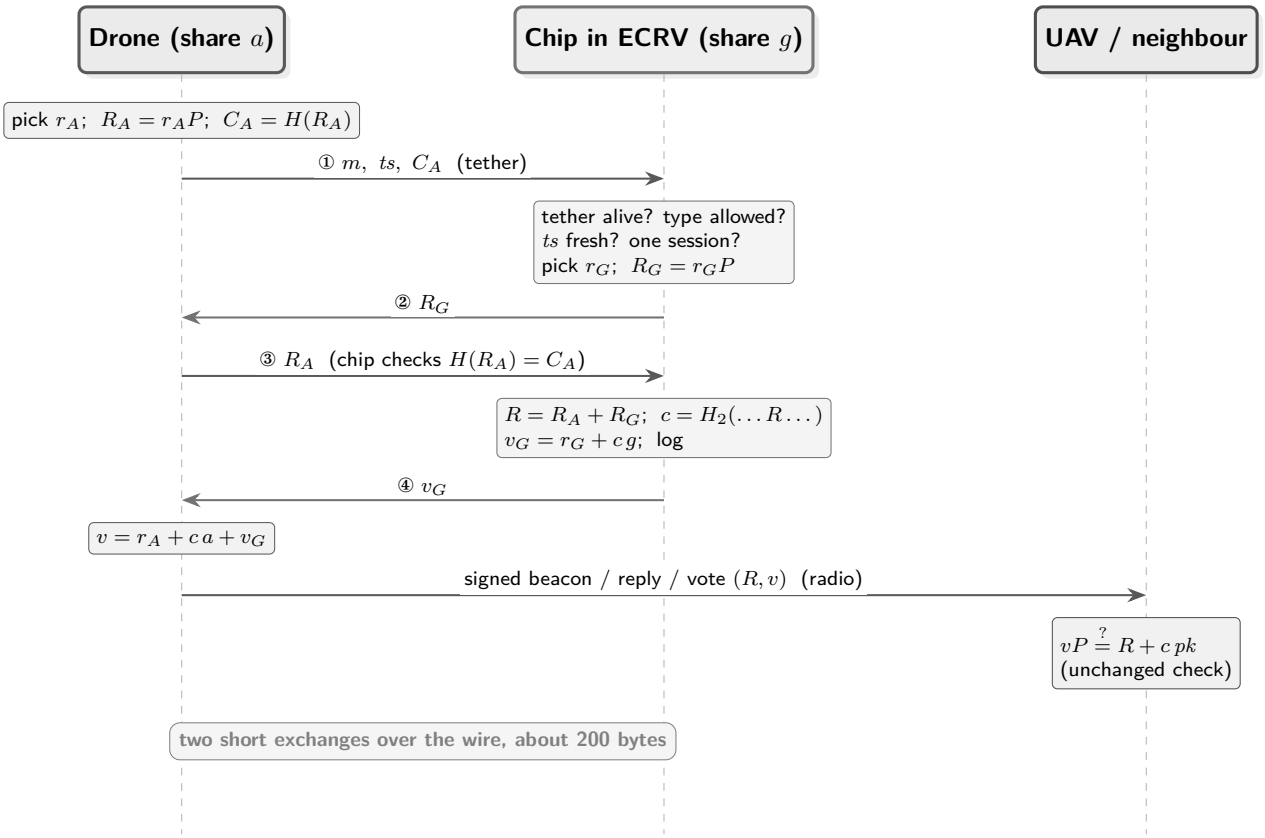


Figure 13.2: Joint signing over the tether (steps J1–J5). Messages ①–④ travel on the tether between the drone and the chip; the final signature (R, v) is sent by radio and is verified with the usual check. The commitment in ① forces the drone to fix its nonce before it sees the chip's.

13.3 Correctness and security

Lemma 13.1 (Correctness)**.** *$aP + gP = pk$, and the jointly produced (R, v) satisfies the ordinary C1 check $vP = R + c\,pk$; the batch equation* (10.1) *is unchanged.*

Proof. $aP + gP = X_A + X_G + dP = X + Y + hP_{\text{pub}} = pk$ (Lemma 10.1). Then $vP = (r_A + c\,a)P + (r_G + c\,g)P = (r_A + r_G)P + c\,(a + g)P = R + c\,pk$. □

Theorem 13.2 (Half a key gives no signing power)**.** *Let $G = gP = pk - aP$. If an adversary that knows the drone share a outputs a valid signature (R, v) on a message the chip never co-signed, then $(R,\ v - c\,a)$ is a valid Schnorr signature under G alone. Hence forging with the drone share is as hard as forging a Schnorr signature under an unknown key, which in the random-oracle model is as hard as the ECDLP (Lemma 2.8).*

Proof. $(v - c\,a)P = vP - c\,aP = R + c\,(aP + G) - c\,aP = R + c\,G$. □

Theorem 13.3 (Share independence)**.** *Because x_G is uniform and secret, the chip share $g = x_G + d$ is uniform from the drone's point of view, and the drone's entire memory determines g only through the public value $G = pk - aP$. Recovering g from a captured drone is therefore the ECDLP for G. The TA cannot sign either: it knows d but neither x_A nor x_G.*

Theorem 13.4 (Capture resistance with past sessions)**.** *Suppose the chip runs J1–J5 with commit-then-reveal and one session at a time. Then even an adversary that controlled the drone side of earlier signing sessions (a software takeover before the capture) and later obtains a cannot sign a message the chip did not approve, unless it solves the ECDLP, in the random-oracle model.*

Proof sketch. Plant an ECDLP challenge as G and simulate the chip without g: because the drone must commit to R_A before seeing R_G, the simulator can read R_A from the random-oracle query behind C_A, choose v_G and c first, set $R_G = v_G P - c\,G$ and program H_2 at $R = R_A + R_G$ – the technique used for three-round Schnorr multi-signatures [7]. A forgery is turned into a signature under G (Theorem 13.2) and forked (Lemma 2.8) to extract g. □

Remark 13.5 (Never use the naive two-round version). If the chip answered many sessions in parallel without the commitment, a hijacked drone could choose its nonces after seeing the chip's and combine sessions to forge – the ROS attack [25]. Use commit-then-reveal with one session at a time, as above, or the standardised FROST protocol [23, 24] if concurrency is needed.

13.4 When the tether is cut

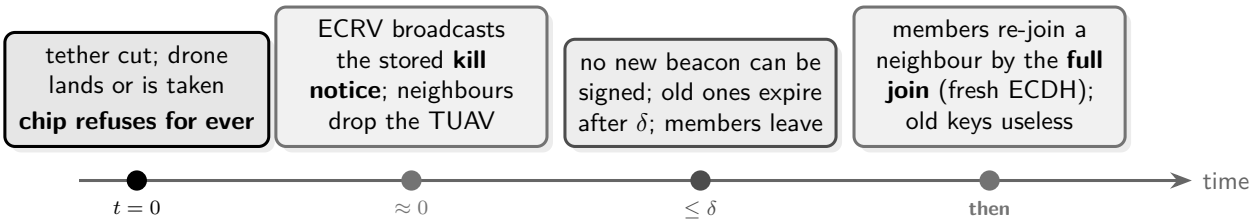


Figure 13.3: Timeline after the tether is cut. The chip stops signing at once, the ECRV broadcasts the stored kill notice, old beacons expire after the period δ, and members re-join a neighbouring TUAV with fresh keys.

13.5 Cost with the base paper's constants

Today one TUAV signature costs $T_{SM\cdot E} + T_{SH} = 0.903\,\text{ms}$. With the split, the drone does $T_{SM\cdot E} + T_{PA\cdot E} + 2T_{SH} = 0.915\,\text{ms}$ (only $+0.012\,\text{ms}$) and the chip $T_{SM\cdot E} + 2T_{SH} = 0.904\,\text{ms}$ *in parallel*. The tether carries about 200 bytes per signature (C_A, R_A, R_G, v_G and m), i.e. 1.6 ms on a slow 1 Mbit/s link. UAVs and neighbouring TUAVs need **no change at all**. The ECRV needs no data centre: a 32-byte share in a SIM-sized chip and one scalar multiplication per TUAV signature.

13.6 What is known and what is new

Splitting a key between a device and a helper that can switch it off is *mediated cryptography* [20]; it was combined with certificateless keys in [21], a pairing-free mediated certificateless signature appears in [22], and two-party and threshold Schnorr signing is standardised as FROST [23, 24]. What is new is *where* we place it: the helper is the ECRV at the other end of the physical tether, the off-switch is the tether itself, and a pre-signed kill notice informs the neighbours. We found no prior work, among the papers citing the base scheme or in the UAV authentication literature, that splits a tethered drone's key across its tether.

Limitations

- If the whole ECRV is seized together with the drone, both shares are lost (a crewed vehicle is much harder to seize than a fallen drone).
- A software takeover while still tethered can still ask the chip to sign within its rules; C3 is the backup.
- The group key in the drone's memory remains exposed until members move to a neighbour and rekey.
- C4 needs a tamper-resistant chip and the careful protocol (commit-then-reveal or FROST); a full formal model of J1–J5 is future work.

CHAPTER 14

A1–A7: attacking our own design

A design that only its authors have looked at is not yet a design. Before finalising C1–C4 we attacked our *own* first version the way a reviewer would, assuming the strongest adversary of Chapter 2: a captured or compromised TUAV. We found seven flaws. These are flaws of our first draft, not of the base paper; the final design described in Chapters 10–13 closes all of them.

	Flaw in our first design	**Fix in the final design**	**Found by**
A1	one forged packet naming a UAV got that UAV evicted (lock-out)	a failed signature never evicts anybody; only valid evidence counts	review
A2	a captured TUAV could mint handover tickets for anybody	the member signs its own handover request (HO2)	review, confirmed formally
A3	a captured TUAV could frame a member at a neighbour	evictions cross cells only with checkable evidence	review
A4	a credential issued for one role could claim another (a member voting as a TUAV)	the role is bound into the certified identity through ep in h	review
A5	a replayed old rekey could roll the group key back	key versions ν may only increase	review
A6	a newcomer received the key already in use (and could read recent traffic)	rekey on every arrival (RK2)	review
A7	a captured old TUAV could pose as the new TUAV in a handover	the new TUAV signs its reply (HO3)	**formal verification**

14.1 Why each fix works

A1 – availability.
A packet with a bad signature proves nothing about the identity it names: anyone can write any pseudonym into a packet. Penalising the *named* UAV therefore lets an outsider lock out any UAV. In the final design an invalid request is simply dropped (Theorem 10.5 guarantees that the valid requests in the same batch still pass).

A2 – a ticket alone is not authority.
The ticket is sealed under K_{ab}, which a captured old TUAV knows. Requiring the member's signature on $(ID_a \,\|\, ID_b \,\|\, H(Tk) \,\|\, E)$ means a ticket admits nobody without the member's own key (Theorem 11.1, property 1).

A3 – evidence, not accusation.
An eviction request from one TUAV is not believed by another cell unless it carries something anyone can verify, such as conflicting messages signed by the accused.

A4 – role binding.
With $h = H_1(PID \,\|\, ep \,\|\, X \,\|\, Y)$ and $ep = 0$ reserved for TUAVs, a member's credential produces a different h and fails V4 of Definition 12.1.

A5 – monotone versions.
An old rekey message is a valid signature on old data; accepting it again would roll the cell back to a key the attacker may know. Members accept a rekey only if its version is larger than the current one.

A6 – backward secrecy for newcomers.
Without a rekey on arrival a newcomer receives the key that protected the last minutes of traffic. RK2 gives it a fresh key instead (Theorem 10.7(c)).

A7 – authenticate both directions.
After A2 the member was authenticated, but the reply was not: a captured old TUAV knows hk and could answer in $\mathcal{T}_b$'s name. The flaw survived our own review and was found only by the symbolic model; with $\mathcal{T}_b$'s signature on $(\ldots \,\|\, E \,\|\, E_b)$ all five properties of Theorem 11.1 hold.

CHAPTER 15

Results: security comparison and cost

15.1 Security properties

Property	Base paper	Our design	Where shown
The issuing authority cannot sign for a UAV (key-escrow resilience)	✗ level 1	✓ level 2	Thm. 7.1, 10.6
Non-repudiation (nobody else could have signed)	✗	✓ (TA excluded)	Cor. 7.2, Thm. 10.4
A removed member cannot read new group keys	✗	✓	Cor. 8.3, Thm. 10.7
A new member cannot read old group keys (backward secrecy)	✗	✓	Cor. 8.4, Thm. 10.7
A passive outsider learns nothing from the broadcasts	✗	✓	Thm. 8.6, 10.7
Forward secrecy after long-term key leakage	✗	✓	Thm. 9.1, 10.8
Login works with the TA offline	✗	✓	Ch. 10
Batch check sound, with a failure path	✗ no weights	✓ 2^{-64} + fallback	Lemma 2.9, Thm. 10.5
Handover between TUAVs	–	✓	Thm. 11.1
Removal of a compromised TUAV without the TA	–	✓ (t-of-n)	Thm. 12.2, 12.3
A captured TUAV cannot impersonate the TUAV	✗	✓	Thm. 13.2, 13.4

15.2 Attacks and their outcome

Attack	Against the base paper	Against our design
TA (or thief of its database) logs in as a UAV	**succeeds** (Alg. 7.1)	fails: needs x_i (ECDLP)
Removed UAV computes the next group key	**succeeds** (Cor. 8.3)	fails: no entry, fresh key
New UAV opens the group's recorded past	**succeeds** (Cor. 8.4)	fails: rekey on arrival
Eavesdropper compares two broadcasts	**succeeds** (Thm. 8.6)	fails: nothing to compare
Thief of a UAV's long-term key opens old sessions	**succeeds** (Thm. 9.1)	fails: CDH on erased exponents
Captured old TUAV mints tickets / poses as the new TUAV	– (no handover)	fails: A2, A7
Captured TUAV keeps acting as the TUAV	**succeeds** (s_{tu} is in the drone)	fails: half a key (Thm. 13.2)

All six attacks that apply to the base scheme succeed against it; the handover attack does not apply, since the base scheme has no handover. Every attack fails against our design, and each failure rests on a theorem.

15.3 Cost

All numbers below are computed from message formats and from the base paper's own MIRACL constants (Section VI-A of [1]); they do not depend on any particular implementation.

	Base paper	Our design
Pairings per login round of n UAVs	$3n$	0
Satellite round trips per login round	1	0
Authenticate and key $n = 120$ UAVs (TA + TUAV)	≈ 3720 ms + satellite trip	≈ 230 ms
Traffic to distribute one group key, $n = 120$	≈ 17.4 MB ($\Theta(n^3)$)	≈ 9.6 KB ($\Theta(n)$)
Extra drone work per TUAV signature with C4	–	+0.012 ms (chip in parallel)
Handover	full re-login through the TA	no TA; saves the beacon wait
Checking an eviction certificate ($t = 2$)	–	≈ 3.6 ms

15.4 What we claim, and what we do not

We claim	We do not claim
three flaws in the base paper, each with a theorem and a proof	that the outsider attack works from a *single* broadcast or for mixed join-and-leave updates
a design that authenticates and keys UAVs with the TA offline	that our TA cannot cheat at all (we reach Girault level 2, not 3)
reduction-based security arguments for C1 and C4, symbolic verification for C2	tight reductions, or security against every possible attack
C2, C3 and C4 as new *uses* of known tools	new cryptographic primitives

CHAPTER 16

Related work

The base paper and its citers. Tan, Zheng and Vijayakumar [1] are, to their knowledge, the first to use a tethered UAV for authentication in infrastructure-less IoVs. We screened the 94 works that cite it; none reports an attack on its CRT key distribution or on its key-escrow claim, and none protects the tether itself.

CRT and polynomial group-key distribution. CRT-based broadcasting goes back to the "secure lock" of Chiou and Chen [10]. Sharma and Purushothama [11] show that prominent CRT-based group-key schemes fail to keep backward secrecy when a user is corrupted; our insider results (F2a, F2b) are an explicit instance of this weakness for the base scheme. Masked-polynomial constructions are known to be fragile in other settings too: Albrecht, Gentry, Halevi and Katz [12] break key-predistribution schemes based on "perturbation polynomials". Our outsider attack (F2c) differs from these: it needs no corrupted user, only two public broadcasts, and it rests on Vieta's formulas. Logical key hierarchies [13] are the standard way to rekey large groups with per-member delivery, and C1 can use them.

Certificateless signatures and batch verification. Certificateless public-key cryptography was introduced by Al-Riyami and Paterson [2] to remove both certificates and key escrow; Girault's trust levels [9] grade how much an authority can still cheat. Our credential is the Schnorr-based identity construction of Galindo and Garcia [3], whose security proof was revisited by Chatterjee, Kamath and Kumar [4], extended with the user's own secret. Schnorr signatures [5] and the forking lemma [6, 7] give the security arguments, and the small-exponent test of Bellare, Garay and Rabin [8] gives sound batch verification.

Handover and tickets. Ticket-based authentication is classic (Kerberos [18]). Handover schemes for UAV and vehicular networks typically assume an authority that remains reachable, or rely on physical unclonable functions or a blockchain. C2 works with the authority offline and binds the ticket to signatures in both directions, which matters precisely when the issuing TUAV may be captured.

Byzantine fault tolerance. C3 borrows the quorum-certificate pattern of PBFT's view change [19]; because eviction is monotone, a quorum of $f + 1$ suffices for safety.

Mediated and threshold signatures. Mediated cryptography [20] splits a key between a user and a security mediator that can revoke it instantly; Chow, Boyd and González Nieto [21] combine it with certificateless keys, and Yap et al. [22] give pairing-free mediated certificateless signatures. Two-party and threshold Schnorr signing is standardised as FROST [23, 24], and the ROS attack [25] shows why naive concurrent signing is unsafe. C4 applies these ideas to a new place: the mediator is the ECRV at the other end of the tether, and the revocation trigger is physical.

Formal verification. We use the symbolic Dolev–Yao model [26] and the ProVerif verifier [27] for the handover protocol.

CHAPTER 17

Conclusion and future work

The base paper takes a good idea – replace the failed roadside unit with a tethered drone – but builds its security on two weak foundations: an authority that knows every UAV's secret, and a broadcast polynomial whose coefficients are symmetric functions of the members' keys. From the paper's own equations we proved three flaws: the TA can impersonate any UAV (F1); the group key leaks to removed members, to newcomers and even to a passive outsider who compares two broadcasts (F2); and one leaked long-term secret opens all past keys (F3).

Our design moves the TA out of the field. Certificateless credentials, Schnorr-type signatures, a sound batch test and per-member sealed key delivery let the TUAV authenticate and key its UAVs alone, and they remove all three flaws (C1). Once the TUAV holds the trust, we protect it: members move between TUAVs with signed tickets (C2), a quorum of TUAVs can evict a compromised one (C3), and the TUAV's key is split across the tether so a captured drone cannot sign (C4). Attacking our own design first (A1–A7) – with formal verification finding one flaw that review missed – made the final version stronger.

The one-sentence summary

The paper cut the cable to the roadside unit but kept a far-away authority and a leaky polynomial inside every key; we take the authority out of the field, seal each member's key separately, and then protect the drone that inherits the trust – by letting other drones vote it out and by splitting its key across the tether.

Future work

- Decide whether a passive outsider can recover the keys from a *single* broadcast, or from one update that mixes joins and leaves.
- A full formal model of the tether-split signing protocol J1–J5, including a hijacked drone that controls past sessions.
- A recovery rule that tells members *which* TUAV to join after an eviction, bound to the same quorum certificate.
- An implementation on real drone and ECRV hardware with a 128-bit curve, and a field test of handover latency.

Bibliography

[1] H. Tan, W. Zheng and P. Vijayakumar, "Secure and efficient authenticated key management scheme for UAV-assisted infrastructure-less IoVs," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 6, pp. 6389–6400, June 2023.

[2] S. S. Al-Riyami and K. G. Paterson, "Certificateless public key cryptography," in *ASIACRYPT 2003*, LNCS 2894, Springer, pp. 452–473, 2003.

[3] D. Galindo and F. D. Garcia, "A Schnorr-like lightweight identity-based signature scheme," in *AFRICACRYPT 2009*, LNCS 5580, Springer, pp. 135–148, 2009.

[4] S. Chatterjee, C. Kamath and V. Kumar, "Galindo–Garcia identity-based signature revisited," in *ICISC 2012*, LNCS 7839, Springer, pp. 456–471, 2013.

[5] C. P. Schnorr, "Efficient signature generation by smart cards," *Journal of Cryptology*, vol. 4, no. 3, pp. 161–174, 1991.

[6] D. Pointcheval and J. Stern, "Security arguments for digital signatures and blind signatures," *Journal of Cryptology*, vol. 13, no. 3, pp. 361–396, 2000.

[7] M. Bellare and G. Neven, "Multi-signatures in the plain public-key model and a general forking lemma," in *ACM CCS 2006*, pp. 390–399, 2006.

[8] M. Bellare, J. A. Garay and T. Rabin, "Fast batch verification for modular exponentiation and digital signatures," in *EUROCRYPT 1998*, LNCS 1403, Springer, pp. 236–250, 1998.

[9] M. Girault, "Self-certified public keys," in *EUROCRYPT 1991*, LNCS 547, Springer, pp. 490–497, 1991.

[10] G.-H. Chiou and W.-T. Chen, "Secure broadcasting using the secure lock," *IEEE Transactions on Software Engineering*, vol. 15, no. 8, pp. 929–934, 1989.

[11] P. Sharma and B. R. Purushothama, "Proving the (in)security of CRT based key management schemes under SAOA model," *Wireless Personal Communications*, vol. 134, no. 3, pp. 1299–1321, 2024.

[12] M. Albrecht, C. Gentry, S. Halevi and J. Katz, "Attacking cryptographic schemes based on 'perturbation polynomials'," in *ACM CCS 2009*, 2009.

[13] C. K. Wong, M. Gouda and S. S. Lam, "Secure group communications using key graphs," in *ACM SIGCOMM 1998*, pp. 68–79, 1998.

[14] A. K. Lenstra, H. W. Lenstra Jr. and L. Lovász, "Factoring polynomials with rational coefficients," *Mathematische Annalen*, vol. 261, pp. 515–534, 1982.

[15] D. G. Cantor and H. Zassenhaus, "A new algorithm for factoring polynomials over finite fields," *Mathematics of Computation*, vol. 36, no. 154, pp. 587–592, 1981.

[16] H. Krawczyk, "Cryptographic extraction and key derivation: the HKDF scheme," in *CRYPTO 2010*, LNCS 6223, Springer, pp. 631–648, 2010.

[17] P. Rogaway, "Authenticated-encryption with associated-data," in *ACM CCS 2002*, pp. 98–107, 2002.

[18] B. C. Neuman and T. Ts'o, "Kerberos: an authentication service for computer networks," *IEEE Communications Magazine*, vol. 32, no. 9, pp. 33–38, 1994.

[19] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *OSDI 1999*, pp. 173–186, 1999.

[20] D. Boneh, X. Ding, G. Tsudik and C. M. Wong, "A method for fast revocation of public key certificates and security capabilities," in *10th USENIX Security Symposium*, 2001.

[21] S. S. M. Chow, C. Boyd and J. M. González Nieto, "Security-mediated certificateless cryptography," in *PKC 2006*, LNCS 3958, Springer, pp. 508–524, 2006.

[22] W.-S. Yap, S. S. M. Chow, S.-H. Heng and B.-M. Goi, "Security mediated certificateless signatures," in *ACNS 2007*, LNCS 4521, Springer, 2007.

[23] C. Komlo and I. Goldberg, "FROST: Flexible round-optimized Schnorr threshold signatures," in *Selected Areas in Cryptography (SAC 2020)*, LNCS 12804, Springer, 2021.

[24] D. Connolly, C. Komlo, I. Goldberg and C. A. Wood, "The Flexible Round-Optimized Schnorr Threshold (FROST) protocol for two-round Schnorr signatures," RFC 9591, IRTF, June 2024.

[25] F. Benhamouda, T. Lepoint, J. Loss, M. Orrù and M. Raykova, "On the (in)security of ROS," in *EUROCRYPT 2021*; also *Journal of Cryptology*, vol. 35, article 25, 2022.

[26] D. Dolev and A. C. Yao, "On the security of public key protocols," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.

[27] B. Blanchet, "Modeling and verifying security protocols with the applied pi calculus and ProVerif," *Foundations and Trends in Privacy and Security*, vol. 1, no. 1–2, pp. 1–135, 2016.